

Prague University of Economics and Business

Faculty of Informatics and Statistics



**MODERNIZING SHIBBOLETH IDP SERVER
USING ANSIBLE AUTOMATION TO ENHANCE
CRISIS MANAGEMENT IN A UNIVERSITY
ENVIRONMENT**

BACHELOR'S THESIS

Study programme: Applied Informatics

Author: Eliška Kosíková

Bachelor's Thesis Supervisor: Ing. Filip Vencovský, Ph.D.

Prague, December 2024

Acknowledgement

I am truly grateful to my supervisor, Ing. Filip Vencovský, Ph.D., for his guidance, support, and valuable feedback throughout the entire process of this work. I also wish to thank my colleagues for their generous support, shared knowledge, and expertise, which greatly contributed to the completion of this thesis.

Abstract

Following thesis aims to modernize the Shibboleth Identity Provider server infrastructure at the Prague University of Economics and Business by implementing Ansible automation. The primary business goal is to enhance the efficiency and reliability of the university's Linux server deployment processes, improve disaster recovery capabilities, and optimize overall server administration and management.

The thesis begins with an analysis of the current state of authentication servers at the university, highlighting the limitations of manual server deployment and upgrades, which lacked standardized procedures and documentation. This approach resulted in inefficient workflows, consuming considerable time and resources. This thesis further explores the role of Ansible in server and application deployment and management and identifies areas that could benefit from automation. It also examines how documentation is used as a code.

Additionally, the thesis details the creation of new Ansible roles designed to automate Shibboleth Identity Provider server deployment tasks. It explains their implementation, evaluates their effectiveness, and demonstrates how the solution contributes to the optimization of server management at the university.

Keywords

Ansible, automation, server administration, disaster recovery, university IT infrastructure, server lifecycle management

Contents

Introduction.....	8
1 Server Automation and Ansible Introduction	10
1.1 Advantages of Linux systems automation	10
1.2 Literature review	11
1.3 Modern Automation Tools	13
1.3.1 Comparative Study of Automation Tools.....	14
1.4 Summary	15
2 Analysis of the Current State of Authentication Servers at VŠE	16
2.1 Introduction and Methodology	16
2.2 Review of Existing Documentation.....	16
2.3 Virtual Machine Server Deployment: Template-Based Approach	17
2.4 Initial State of the Authentication Servers	19
2.4.1 Shibboleth Server Configuration Analysis.....	19
2.4.2 Best Practices Alignment	21
2.4.3 Version Compatibility	21
2.5 Disaster Recovery Plan Assessment	22
2.5.1 Current DRP Overview	22
2.5.2 Gaps and Risks	23
2.6 Server Lifecycle Management with Ansible.....	24
2.6.1 Different Approaches to Server Management – PETS VS. CATTLE.....	24
2.6.2 Practical Applications in Our Environment.....	25
2.7 Opportunities for Automation and Improvement	25
2.8 Summary.....	27
3 Plan for proposed improvements and automation	28
3.1 Preparation based on known configuration	28
3.1.1 Initial setup role	29
3.1.2 Creation of roles.....	31
3.2 Ansible Architecture	31
3.3 Documentation as Code	32
3.4 Configuration Management and Role Standardization.....	34
3.5 Summary	35
4 Implementation	36
4.1 Playbook	36

4.1.1 Ansible vault.....	37
4.2 Testing and Validation Framework	38
4.3 Time Efficiency and Productivity Gains	39
4.4 Updates and Security Patch Management.....	40
4.5 Disaster Recovery Automation	41
4.6 Future Development and Scalability Considerations	42
4.7 Summary	42
Conclusions.....	44
List of references	45
Appendices	I
Appendix A: Ansible Role Documentation	I

A List of Programme Code Excerpts

Code 1 Documentation of Initial setup role written as code (Source: author).....	33
Code 2 Example of using variables within Jinja2 template (Source: author).....	34
Code 3 Playbook for id servers (Source: author).....	37
Code 4 Section of host_vars file for id servers (Source: author).....	38
Code 5 Structure of initial_setup role (Source: author).....	I
Code 6 Main.yml file in initial_setup role (Source: author)	II
Code 7 set_hostname.yml (Source: author).....	II
Code 8 set_ntp.yml (Source: author)	III
Code 9 upgrade_packages.yml (Source: author)	III
Code 10 install_basic_packages.yml (Source: author)	IV
Code 11 install_additional_packages.yml (Source: author).....	V
Code 12 setup_autoupdate_apt.yml (Source: author)	V
Code 13 setup_bash.yml (Source: author)	VI
Code 14 secure_ssh.yml (Source: author)	VII
Code 15 Handlers in initial_setup role (Source: author)	VIII
Code 16 Structure of java role (Source: author)	IX
Code 17 Main.yml file in java role (Source: author).....	IX
Code 18 install_java.yml (Source: author)	IX
Code 19 set_java_home.yml (Source: author).....	X
Code 20 mark_hold.yml (Source: author)	X
Code 21 Structure of tomcat role (Source: author)	XI
Code 22 Template in tomcat role (Source: author).....	XII
Code 23 Main.yml file in tomcat role (Source: author).....	XII
Code 24 install_tomcat.yml (Source: author).....	XIII
Code 25 create_instances.yml, first part (Source: author)	XIV
Code 26 create_instances.yml, second part (Source: author)	XIV
Code 27 adjust_configuration.yml (Source: author)	XVI
Code 28 start_tomcat.yml (Source: author)	XVI
Code 29 Handlers in tomcat role (Source: author)	XVII
Code 30 Structure of shibboleth role (Source: author)	XVIII
Code 31 Main.yml file in shibboleth role (Source: author)	XVIII
Code 32 prepare_system.yml (Source: author)	XIX
Code 33 install_shibboleth.yml (Source: author)	XIX
Code 34 modules_and_plugins.yml, first part (Source: author).....	XXI
Code 35 modules_and_plugins.yml, second part (Source: author)	XXII
Code 36 Templates in shibboleth role (Source: author)	XXIII

List of Abbreviations

IC	Informatics Centre
VŠE	Vysoká škola ekonomická v Praze/ Prague University of Economics and Business
IdP	Identity Provider
SSO	Single-Sign On
DRP	Disaster Recovery Plan
SSH	Secure Shell
IaC	Infrastructure as Code
DSS	Department of System Support
IP	Internet Protocol
VMs	Virtual Machines
OS	Operating System
IPAM	IP Address Management
NTP	Network Time Protocol
FQDN	Fully Qualified Domain Name

Introduction

The need for modern, efficient, and resilient server infrastructure has become increasingly significant in both corporate and academic environments. The continuous demand for innovation and development in server administration at my place of employment, the Informatics Centre (IC) of the Prague University of Economics and Business (Vysoká škola ekonomická v Praze, VŠE), motivated me to focus on a project to automate server deployment and management. In line with the department's specialization, I will focus on servers running the Linux operating system. Among many servers we maintain at the IC we also take care of the authentication servers, specifically Shibboleth Identity Provider (IdP) servers. Shibboleth is a widely used authentication system that enables identity federation and single sign-on (SSO) across institutions. Further explanation of Shibboleth's functionality is beyond the scope of this thesis, as this work focuses primarily on the automation with Ansible. I decided to explore the use of Ansible automation on these servers because they are currently due for a significant update, and their deployment and administration rely on many manual tasks. The Ansible roles and playbooks I will develop as part of this project will not only enhance their specific deployment and management processes but will also be generalized for use in other server environments. This will guarantee their flexibility in comparable situations, benefiting our team as well as others who may face similar challenges.

The main goal of this thesis is to modernize the Shibboleth IdP server infrastructure at VŠE by implementing Ansible automation, with the aim of enhancing server deployment efficiency, strengthening disaster recovery processes, and optimizing server lifecycle management. Given the increasing complexity of managing authentication systems, a more automated, standardized method might significantly improve both the efficiency and reliability of crucial server operations. While functional, the current system at our university lacks essential automation and comprehensive disaster recovery features that meet modern standards of reliability and efficiency. Many server management tasks are performed manually, which introduces the potential for human error and leads to inefficiencies. This research addresses the gap in automation by proposing the use of Ansible to improve the server lifecycle management and deployment speed and strengthen disaster recovery procedures.

This study is significant as it provides a practical solution to the growing issues encountered by server administrators, not only in academic settings and in IC but in other sectors as well. Automating server administration with Ansible can reduce the workload for IT professionals while simultaneously improving the resilience and stability of important university systems.

The methodology used in this thesis will be primarily qualitative. First, a literature review will be conducted, focusing on Ansible automation and its applications in similar environments, as well as modern approaches to server automation and documentation as a code. I aim to provide the reader with a clear understanding of how Ansible functions as a

powerful tool for automation. By examining existing theories and documentation, I will highlight its advantages and various approaches to server management.

Following the literature review, I will analyse the internal documentation and server configuration files and conduct unstructured interviews with key colleagues to evaluate the current condition of the Shibboleth servers, deployment, and management procedures, and identify areas for improvement. This process will include an assessment of the existing manual tasks performed regularly, as well as an examination of the current backup and recovery procedures. The qualitative method of unstructured interviews with two colleagues will be essential to this analysis, offering insights into the decision to use Ansible and the present state of the university's authentication servers and disaster recovery plan (DRP). This will allow me to meet the first secondary objective of documenting the current structure and server administration processes, while also assisting me in identifying areas for improvement, forming the second secondary objective.

In the planning phase, I will outline the creation of necessary Ansible roles and tasks, focusing on designing generalized roles for the deployment of authentication servers, which form the third objective. To ensure future usability, I will document these roles as code. During the implementation phase, I will combine these roles into a functional playbook to facilitate efficient server setup and management.

After implementation, I will evaluate the effectiveness of the Ansible solution. In the evaluation phase, quantitative methods will be used to measure the impact of Ansible automation. The key metric will be task completion times, which will be compared between manual and automated processes. This will allow for an objective assessment of the improvements in efficiency, reliability, and disaster recovery capabilities resulting from the automation.

Finally, in the concluding chapter, I will evaluate the results, and the improvements achieved against the defined objectives and summarize the outcomes.

1 Server Automation and Ansible

Introduction

Server automation refers to the use of tools and scripts to automate and mechanise repetitive tasks related to server administration. As IT infrastructures become more complex, server automation assists administrators in the effective management of servers by eliminating the need for manual intervention in operations such as configuration, data management, provisioning, monitoring, and security enforcement (Campi & Bauer, 2009, s. 10).

1.1 Advantages of Linux systems automation

One of the most significant advantages of using automation is decreasing the time dedicated to routine jobs performed daily, including software updates, ensuring system security, and managing server configurations (Campi & Bauer, 2009, p. 7). Nevertheless, even less frequently performed procedures might benefit from automation, as they are more susceptible to human mistakes because administrators may forget the process of the task over time or require additional time to reacquaint themselves with the routine. Automation facilitates the execution of commands across numerous servers simultaneously, eliminating the need for manual login each time, therefore assuring consistency, minimising human error, and decreasing the time required for the task to be finished.

Server automation is particularly useful in scenarios where high availability and low downtime are required. System upgrades, software installations, and compliance checks can be scheduled to execute automatically at certain times, some of which may be during off-peak hours, to reduce disturbances. For example, such tasks can be scheduled database backups or software updates running overnight (Ghalib, 2021). This is critical for ensuring an efficient and secure infrastructure while minimising operational time and costs.

By focusing on automating Linux server tasks, system administrators can ensure that their servers remain secure, up-to-date, and compliant with organizational standards. Some of the key tasks that can be automated include:

- **Installing new systems** - deploying new servers using either pre-defined or specific variable configurations dependent on the client's requests.
- **Updates, upgrades and patching** - applying system updates and security patches to ensure vulnerabilities are fixed as soon as possible.
- **Managing configuration** - maintaining consistent server configurations across a large number of systems, ensuring that any modifications are applied uniformly.

- **Monitoring and alerts** - continuously tracking system health and performance parameters, with automated alerts sent when thresholds are exceeded (Campi & Bauer 2009, p. 11).
- **Backup and recovery** - guaranteeing that data backups are performed on a regular basis, and that disaster recovery protocols are in place.

By automating these operations, administrators may redirect their attention away from time-consuming manual interventions and towards more strategic tasks that contribute to the overall development and stability of their IT infrastructure.

Regardless of the size of your infrastructure, implementing automation is always advantageous. Even if you only have a single Linux system, automating routine tasks can save time and ensure essential operations are completed consistently and on schedule. Even a single server can encounter challenges like overflowing storage, which can result in costly downtime or data loss (Campi & Bauer, 2009, p. 3). Automation acts as a precaution, making sure that the tasks required to fix the issue are handled promptly, ultimately leading to better management of resources and more resilient infrastructure. Furthermore, if your system requires replacement due to hardware failure or updates, automation may assist in reconfiguring it to the same standards as previously. And that is slightly giving us a clue as to how automation might help in DRP.

1.2 Literature review

I completed thorough literature research for my thesis using several academic databases and internet search engines. Google Scholar and Google Books were my primary sources of information, in addition to the scientific article and book databases available through Prague University of Economics and Business's library, such as the catalogue and e-resources, as well as the Summon and Springerlink search engines.

I started by looking for theoretical materials about automation in general and how it has evolved over the last few decades. My original search query was "automate linux," which led me to a very informative book called *Automating Linux and Unix System Administration* (Campi & Bauer, 2009). I borrowed this book from the National Library of Technology. It offers a foundational understanding of automation principles, using tools like Cfengine, Bash, and Perl. In the first chapters it lays groundwork for understanding the evolution of automation, discussing its early stages and how these concepts have become integral to server management over time. I reference these chapters, which cover general automation concepts, in chapter 1.1 of this thesis.

Subsequently, I modified my search term to "history of server administration automation" and found another valuable source *Modern System Administration: Managing Reliable and Sustainable Systems*, which covers modern tools and practices, including version control, virtualization, containers, and infrastructure management, while focusing on essential topics like monitoring, observability, and security (Davis, 2022).

Since we had been experimenting with Ansible at the IC for some time before deciding to fully adopt it for larger-scale automation, we already had an inclination toward it. However, I still wanted to explore other automation tools for comparison. To begin my research, I searched the term "how to automate IT infrastructure" in Google Books and found *Automating Infrastructure with Terraform and Ansible* (Phoetean, 2024). It goes deeper into using Terraform for infrastructure provisioning and Ansible for configuration management, emphasizing how these two tools may collaborate to automate infrastructure at scale. I overviewed the practical examples and insights about managing cloud infrastructure efficiently, especially when adopting infrastructure-as-code principles. I realized it would be more practical to investigate recent studies, theses, and research papers on Ansible to get more insight into this topic.

I used the Summon database and the VŠE catalogue to search for the term “ansible” and I found exactly what I was looking for, which was a comparison of automation tools as students have already conducted similar research. I discovered three theses. One is *Using the Ansible platform in IT infrastructure automation*¹ which focuses more on creating a methodology for using the Ansible in IT infrastructure automation called MAIPA (Olexová, 2020). The author of that thesis clearly explains the differences between various automation tools, along with the characteristics and principles of Ansible, using well-researched and credible sources. Another valuable resource was *Graphical user interface for managing Ansible system configurations*² (Jochim, 2023). While the creation of the graphical interface itself wasn't as relevant, the practical insights into Ansible were informative.

While the mentioned theses inspired me to explore additional sources, I continued my own research and found several other theses related to Ansible. However, these were less relevant, dealing with automating different types of systems or working within different operating systems. Though not directly aligned with my focus, they provided insights into how Ansible is applied in various contexts.

In addition to academic literature, I have mainly relied on the official Ansible documentation at docs.ansible.com, maintained by the Ansible project contributors and supported by Red Hat, Inc. (last updated 2024). This documentation serves as a comprehensive guide to understanding and utilizing all aspects of Ansible. Although some of the authors of the aforementioned theses expressed concerns about the lack of available resources at the time, Ansible has since become widely used along with other automation platforms, especially in the deployment processes, which we can see in Figure 1.

¹ The original title of the thesis in Czech is *Využití platformy Ansible pro automatizaci IT infrastruktury*. This is the official English translation.

² The original title of the thesis in Czech is *Grafické uživatelské rozhraní pro správu konfigurací systému Ansible*. This is the official English translation.

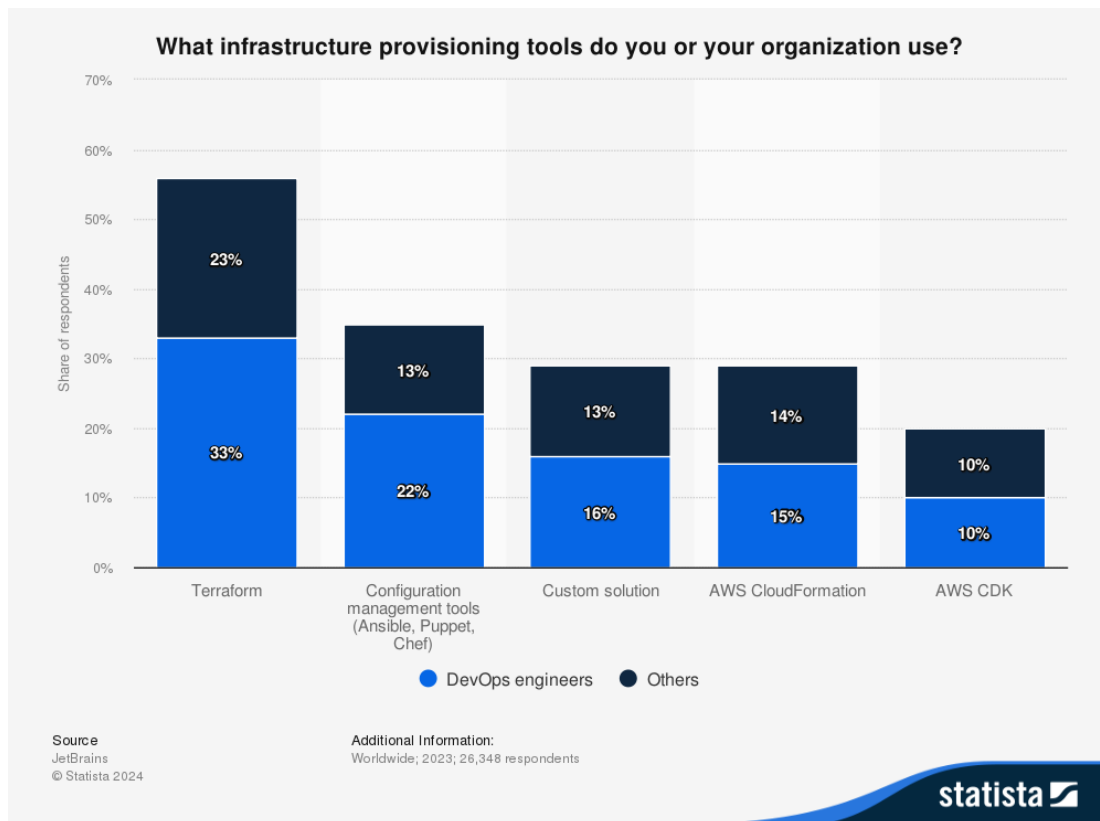


Figure 1 What infrastructure provisioning tools do you or your organization use? (JetBrains, 2023)

Growing popularity has led to an abundance of material, including books, video tutorials (some from Red Hat itself) and thorough resources for troubleshooting, configuration, and modules that make it easier to master common tasks. I did find lots of books and guides when searching for “ansible” and for “documentation as code”.

Finally, I would like to mention the databases where I did not retrieve relevant results while searching for phrases like “ansible”, “server automation”, and “IT infrastructure automation”. When trying to find research papers on Arxiv.org, the search returned very few results with little relevance. Similarly, a search on the Digizeitschriften site showed no results. The Springerlinks site offered mostly general tutorials, with one mention of Ansible for VMware, but other databases such as the ACM Digital Library, Scopus, Web of Science, and EZB provided no relevant results.

1.3 Modern Automation Tools

Scripting languages such as Bash, Perl, and Python remain essential, but the complexity of modern IT infrastructure demands more advanced, scalable solutions. *While Bash would continue to be one of the basic requirements of every sysadmin, one needs to leverage his/her skills into real automation beyond shell scripting* (Alsadi, 2017). Today, tools like Ansible, Saltstack, Puppet, and Chef have transformed automation by providing more advanced features. These platforms allow users to manage configurations, deploy infrastructure, and perform continuous integration with minimal manual intervention.

1.3.1 Comparative Study of Automation Tools

In one of the theses reviewed in the literature review chapter 1.2, Aayushi Johari's comparison table of configuration management technologies is referenced (Olexová, 2020, p. 26) to highlight Ansible's primary benefits, particularly in terms of simplicity and flexibility (see Table 1). This aligns with solutions from earlier theses, which showed that Ansible was the preferred solution for various automation use cases, including Automation Using Ansible in Windows (Klíč, 2018) and Network configurations (Hudec, 2016), due to its simplicity of setup and administration. However, it should be noted that these two works did not consider other automation platforms or did not clarify why Ansible was chosen.

Table 1 Chef vs Puppet vs Ansible vs Saltstack: Which Works Best For You? (Johari, 2024)

METRICS	CHEF	PUPPET	ANSIBLE	SALTSTACK
AVAILABILITY	✓	✓	✓	✓
EASE OF SETUP	Not very easy	Not very easy	Easy	Not very easy
MANAGEMENT	Not very easy	Not very easy	Easy	Easy
SCALABILITY	Highly Scalable	Highly Scalable	Highly Scalable	Highly Scalable
CONFIGURATION LANGUAGE	DSL(Ruby)	DSL(PuppetDSL)	YAML(Python)	YAML(Python)
INTEROPERABILITY	High	High	High	High
PRICING (UP TO 100 NODES)	\$13700	\$11200-\$19900	\$10,000	\$15,000(approx.)

In addition to the table comparison, the study (Olexová, 2020, p. 27) includes a compelling graph demonstrating Ansible's rising popularity between 2012 and 2017 (Johari, 2024). This trend reflects Ansible's rising popularity in the DevOps community, with an emphasis on its versatility and ease of use for server administration. Later theses also explore the differences among these automation platforms, emphasizing Ansible's agentless approach (meaning there is no additional software that needs to be installed on controlled nodes and communication occurs over Secure Shell (SSH)) while other platforms are agent-based (Jochim, 2022, p. 27). The author also mentions Terraform as a complementary tool for Infrastructure as Code (IaC) provisioning (Jochim, 2022, p. 25). Integration of these tools for streamlined deployment and management is highlighted in Deployment and management of applications in Cloud using Ansible and Terraform (Bielik, 2020), but the thesis focuses primarily on cloud environments, specifically AWS.

When comparing popular automation tools like Chef, Puppet, Ansible, and SaltStack, each provides strong availability features with built-in solutions to ensure failover. However,

their approaches and usability differ, especially in configuration languages, agent requirements, and scalability, making some tools more suitable for certain environments than others.

Chef and Puppet use Ruby-based languages, which are more complex and generally require more programming knowledge. In contrast, Ansible and SaltStack use YAML, which is simpler and easier for system administrators to learn. This simplicity is supported by a large developer community (GitHub, Inc., n.d.), contributing to its popularity for configuration management, especially in cases where straightforward setup and management are essential.

All four tools have robust scalability, though performance varies. Ansible's architecture makes it highly suitable for smaller-scale projects, but it may be slower than competitors when managing large numbers of nodes due to its lack of a built-in mechanism for handling parallel tasks at scale. Conversely, Chef and Puppet are often preferred for extensive automation projects in production, particularly when managing large infrastructures. Chef's infrastructure-as-code model and Puppet's declarative approach allow configurations to define end-states directly, which can be beneficial for maintaining stability across numerous nodes. SaltStack, with its fast, event-driven architecture, excels in environments that require real-time responses, such as network automation projects.

All tools are available in open-source and free versions, but premium features and support are offered at various price points by their respective companies. While competitors are more expensive and complex, Ansible offers a cost-effective solution, referring specifically to Ansible Tower, which is an enterprise-level framework built on top of Ansible, developed by Red Hat, Inc. (Olexová, 2020). My thesis focuses on the community version of Ansible, which is free to download and use. Organizations can test this version before deciding whether to invest in Ansible Tower, a more comprehensive product with a user-friendly GUI, or stick with the free option for simpler needs (Olexová, 2020).

1.4 Summary

First chapter has explored the role of server automation in streamlining day-to-day IT tasks, improving disaster recovery techniques, and ensuring system reliability. Automation allows operations that would otherwise require manual intervention to be completed effectively, reducing human error and enhancing total system uptime.

Based on the comparative study of automation tools, along with the literature review, it is evident that tools like Puppet, Chef, and SaltStack have their strengths. But our research and hands-on experience with Ansible indicate that it best meets our requirements. Ansible's agentless architecture, simple syntax, and flexibility in automating across various platforms (e.g., Windows, networks, cloud infrastructure) make it a practical choice. Given our existing familiarity with Ansible, it aligns perfectly with the goals of this project, offering both scalability and simplicity for managing the university's diverse IT Linux infrastructure.

In conclusion, server automation is essential for improving operational efficiency, and based on the research conducted, Ansible will play a key role in implementing effective and flexible automation processes in our university environment.

2 Analysis of the Current State of Authentication Servers at VŠE

2.1 Introduction and Methodology

In this chapter, the initial state of the authentication servers at VŠE is examined, with a primary focus on the Shibboleth IdP. The objectives of this analysis are to evaluate the existing infrastructure and server deployment procedures, compare them against best practices, and identify areas for improvement, particularly in crisis management and automation.

Building on the methodology outlined in the introduction, the analysis begins with a detailed review of both internal and external documentation, including Václav Pleskač's bachelor's thesis, which provides foundational insights into the Shibboleth IdP servers. Internal sources, such as configuration files and DokuWiki (the open-source wiki application used within the Department of System Support, DSS), were also carefully examined to gain a deeper understanding of the technical aspects of the current deployment. To complement this, unstructured interviews were held with colleagues responsible for server management to gather practical insights and firsthand operational experiences. These interviews were conducted at the VŠE in the IC DSS's offices, following the analysis of the internal documentation, which enabled me to frame targeted questions and better contextualize the responses.

Additionally, a comparison of the collected data against the relevant official documentation was made (for Shibboleth, Tomcat and Debian). This step was crucial in identifying any gaps or deviations from established best practices.

The findings from this comprehensive analysis lay the groundwork for recommending improvements in server automation, standardization, and deployment processes proposed in subsequent chapters.

2.2 Review of Existing Documentation

Automating the deployment of a new Shibboleth IdP server begins with a thorough understanding of the current server environment and the manual tasks typically involved. Given my limited experience with manual IdP server deployments, my first step was to investigate the existing system setup and identify the processes that could be translated into automated workflows. Concurrently, the university's Shibboleth servers are undergoing a significant upgrade between versions, requiring careful review of changelogs and configuration changes to ensure a seamless transition.

A key resource in this process has been Václav Pleskač's bachelor's thesis, which provides valuable insights into Shibboleth IdP server setup and its configurations (Pleskač, 2019). Additional guidance for specific configurations was drawn from the official Shibboleth documentation. I also retrieved information from our department's internal knowledge base on DokuWiki, where critical process descriptions are stored. These resources ensure that all members of the DSS have access to detailed instructions on operational tasks. However, the structure of the documentation is fragmented, with inconsistent levels of detail, which posed challenges in piecing together a complete understanding of the deployment process.

My exploration of the DokuWiki began with the key page dedicated to the IdP `id.vse.cz` server. This page provides an overview of the current Shibboleth IdP setup, including server roles, Internet Protocol (IP) address assignments, firewall configurations, and procedures for the most frequently performed tasks. It also outlines the crisis plan, detailing steps such as restoring lost or corrupted configuration files using Btrfs snapshots, switching master servers in the event of a sudden master server failure, or fully recovering from server loss by cloning a server from an existing template (Department of System Support, n.d.). While these instructions appear thorough, they rely heavily on manual intervention and familiarity with undocumented nuances, increasing the risk of potential human error.

Further review of the DokuWiki also revealed a significant gap in documentation regarding server deployment tasks, such as initial configurations, security protocols, and version control. This lack of detailed procedural documentation became apparent during conversations with colleagues, who are highly experienced in these tasks and suggested that *I draft a proposal for standardizing the process of creating new VMs*³ (DSS, personal communication, February 13, 2024). As a result, I began by compiling a comprehensive list of the tasks required for server deployment. Since these tasks are repetitive and time-consuming, automating them would significantly enhance efficiency and ensure consistency across deployments.

2.3 Virtual Machine Server Deployment: Template-Based Approach

The Shibboleth IdP servers at VŠE are hosted on Virtual Machines (VMs), making virtualization a critical aspect of their deployment and management. Virtualization provides flexibility, scalability, and consistency across deployments. To streamline the provisioning process, VMs at VŠE are created using a template-based approach in VMware.

A VM template is a master copy image⁴ of a virtual machine that includes VM disks, virtual devices, and settings. A VM template can be used many times over for the purposes of VM

³ All personal communications (unstructured interviews) cited in this thesis were translated by the author from Czech to English.

⁴ Also commonly referred to as a "golden image" in the industry.

cloning. You cannot power on and edit the template once it has been created. This is by design, so that nobody can accidentally edit the virtual machine that is used as a template. (NAKIVO Team, 2023)

For all our virtual servers, we rely on a general, minimally pre-configured template running the Debian operating system (OS), recently upgraded to Debian 12 (codenamed Bookworm). This template-based deployment strategy allows for quick deployment of new Debian OS servers. The template is regularly updated with the latest security patches and performance improvements. The VM template used at our department was created by installing the Debian OS, applying the necessary updates, and converting the VM into a template, with CPU and memory configurations set within VMware. Additionally, the template includes pre-configured SSH keys for root access to streamline secure connections during deployment.

However, after deploying a new virtual server from this template it was still required to manually configure and initially set up the VM for each server. These tasks included updating the system, configuring security measures, setting up IP/network, and installing required software. While these tasks followed a similar process, they involved different values for each server, which were considered as variables.

Manual Setup Tasks

This is the list of several tasks that used to be performed manually to prepare each new server, starting with verifying and updating the Debian12_template as follows:

1. Registering a new server:

In the IC's Sauron system (IP Address Management, IPAM), a new host is added by allocating a unique IP address, hostname, and additional information in the required zone and network, ensuring clear and organized records.

At the VM level, the IP address is changed to the allocated one by modifying configuration files such as /etc/hosts and /etc/network/interfaces.

2. Firewall setup:

FortiGate configuration: The network is divided into multiple VLANs, requiring the addition of appropriate rules on the segmentation firewall to allow inbound traffic based on the server's role.

Server-side firewall: Iptables settings for added security are adjusted.

3. Additional tasks:

- Install essential packages and software required for the server's operation.
- Configure the hostname and set up Network Time Protocol (NTP) for time synchronization.
- Customize the bash shell for ease of use.
- Install and configure Xymon for server monitoring.

- Install Legato packages and perform necessary configurations for backup.

These initial tasks, while necessary for a stable and secure deployment, were time-consuming and prone to human error. Documenting these steps thoroughly was the first step toward automation, which allows us to streamline these processes and maintain consistency across deployments.

As part of this automation process, one of the first steps was to create a general Ansible role that can be used to automate some of these processes. *Now that you know the tasks and how to execute them, it would be beneficial to prepare a general Ansible role for the initial setup process that can be used for all of our new servers* (DSS, personal communication, February 20, 2024). This role will be designed for flexibility, making it applicable not only for virtual servers but also for physical machines.

When deploying new physical servers, there are additional steps to consider, as physical servers don't have a pre-configured template like the one we prepared for our virtual servers. For instance, tasks such as disk partitioning and formatting and BIOS or firmware updates are typically required for physical servers. Often, we need to mount partitions based on server needs, typically configuring a primary partition for the operating system and additional partitions for storage.

2.4 Initial State of the Authentication Servers

The goal of this section is not merely to explain the specific details of how the server works, but rather to provide an example of how I analyse the current setup to identify areas that can be automated. In order to effectively automate processes, it is crucial to first understand what is currently on the server, how it operates, and where automation could streamline workflows, improve efficiency, and reduce manual intervention. This approach involves a thorough review of the existing configuration, identifying opportunities for automation, and comparing the setup against best practices.

2.4.1 Shibboleth Server Configuration Analysis

As advised during the interview, one priority is to *look at the requirements for the newest stable release V5 of Shibboleth IdP to which we plan to upgrade, compare them to the current configuration, check out the changelogs, and make sure the transition will be smooth* (DSS, personal communication, March 8, 2024). This recommendation underscores the importance of evaluating Shibboleth V5 system requirements against our current setup with Shibboleth 4.0.1, ensuring compatibility and an efficient transition.

Our existing configuration relies on Tomcat as the servlet container (currently version 9.0.31), supported by Debian's OpenJDK 11. According to the Shibboleth official documentation, a servlet container implementing Servlet API 5.0 or above is necessary for proper operation. Shibboleth explicitly mentions Jetty 11+ and Tomcat 10.1+ as suitable servlet containers that comply with this requirement and OpenJDK 17 or newer for V5 (Cantor, 2024).

Choosing Between Jetty and Tomcat

Shibboleth recommends using Jetty as the preferred container implementation. They state that *all development and most testing time by the core project team is confined to the Jetty platform* (Cantor, 2024). When it comes to choosing between Jetty and Tomcat as servlet containers for Java-based web applications, several factors play a decisive role. Jetty is lightweight and easy to integrate, making it an optimal choice for scenarios where low resource consumption and high performance are priorities. Its asynchronous functions and efficient handling of concurrent requests are well suited to applications requiring non-blocking I/O or high concurrency. Tomcat, on the other hand, is stable, mature and has a wide ecosystem, making it a preferred option for deploying standard Java web applications. Its comprehensive documentation, broad user base and compatibility with a variety of tools and extensions make Tomcat a reliable and easy-to-integrate choice. *On whole, the key difference between Tomcat and Jetty is the fact that Apache Tomcat places great stress on being up to date with the latest specification, while Eclipse Jetty prioritizes the needs of their user community which tends to result on an emphasis on performance* (McKenzie, 2019). Since Tomcat versions 10.1 and above comply with the Servlet API 5.0 requirement, which is necessary for Shibboleth and considering our prior experience, we may choose to proceed with this option.

Existing Services and Configuration

The decision for Tomcat also supports the fact that we incorporate Tomcat's system architecture for handling various services in the server setup, including authentication processes, logging, and external applications.

System Architecture: Applications currently run on a btrfs subvolume with Snapper for filesystem recovery. However, based on internal discussions (DSS, personal communication, November 28, 2024), this approach is planned to be replaced in favour of provisioning new servers using Ansible for automation.

Systemd Units: Applications are managed by dedicated systemd units (tomcat-shibboleth for shibboleth operations itself and there is also another application which will not be covered in this thesis).

Logging

Tomcat's logging configuration is well-managed across different parts of the system. Tomcat itself uses logging.properties to ensure that logs older than 90 days are automatically deleted. This configuration ensures that logs do not consume excessive disk space and aligns with Tomcat's documented best practices for maintaining a lightweight and efficient server environment. The configuration extends to the server.xml file, where an AccessLogValve is configured to log HTTP requests and manage the lifecycle of access logs through the maxDays attribute, which limits the retention of logs to 90 days. Additionally, according to security standards, all logs are immediately forwarded to a central log management platform for storage and analysis, reducing the risk of data loss and enhancing system

monitoring. The Shibboleth IdP logs, on the other hand, follow a slightly different approach due to the nature of its integration with Logback.

Monitoring and Maintenance

System monitoring is a critical component of maintaining operational stability and security. The servers are centrally monitored using Xymon, which ensures that all required services are running as expected, in the correct locations, and that the system's performance metrics (such as CPU, RAM, and disk space) remain within acceptable limits. This approach adheres to standard security practices and ensures prompt detection and resolution of potential issues.

2.4.2 Best Practices Alignment

To ensure a smooth transition to Shibboleth IdP 5 while maintaining best practices, I referred to the official Shibboleth documentation for configuring Tomcat as the servlet container. The documentation emphasizes a modular approach, with a clear separation of Tomcat's core binaries (CATALINA_HOME) from configuration files specific to the Shibboleth IdP instance (CATALINA_BASE). It also provides guidelines for configuring essential components such as HTTP/HTTPS connectors, logging, and deployment of the IdP application (Zeller, 2024). These best practices informed the design of the Ansible roles and templates, ensuring compliance with the latest standards and efficient implementation of Shibboleth IdP in the university environment.

2.4.3 Version Compatibility

The current environment, running Shibboleth 4.0.1, must be majorly upgraded to to the latest stable version (Shibboleth 5.1.3) available at the time of this work to maintain compliance with modern standards and take advantage of significant improvements. Some of the enhancements in Shibboleth IdP 5.x include:

- Expanded REST API capabilities: Greater flexibility for managing accounts and sessions.
- Enhanced metadata resolution and caching: Improved performance and efficiency across restarts.
- Improved cryptographic support: Strengthened security for metadata handling and authentication processes.
- Revised configuration management: Expanded use of the Module feature to better track and manage configuration files, ensuring version control and easier upgrades.

The transition to Shibboleth 5.1.3 introduces notable challenges, such as:

- Upgrading the servlet container: The move to Tomcat 10.1+ is required to meet updated Servlet API 5.0 requirements.
- Adapting to new configurations: Changes to configuration files, namespaces, and defaults necessitate careful testing and validation to ensure compatibility.

- Testing third-party extensions and customizations: Custom scripts and Java extensions must be reviewed and adjusted to align with updated APIs and dependencies.

Looking ahead, Shibboleth's rapid evolution highlights the need for a flexible and systematic approach to managing updates. Future versions may bring similarly significant changes to default configurations, module management, and dependency requirements. Preparing for these updates will be crucial to minimizing downtime and ensuring operational stability.

There have been a lot of changes in the configuration files of Shibboleth between versions 4 and 5. Ansible's configuration management features, particularly dynamic templating, will allow us to maintain this configuration in a single location and easily apply updates as system requirements evolve in the future too. This centralized approach supports efficient adaptations for both Shibboleth V5 compatibility and future maintenance, reducing the risk of configuration drift across environments.

We can also create reusable roles and templates to streamline shibboleth deployment. This approach not only reduces the risk of manual errors during migrations but also establishes a scalable solution for handling future updates to Shibboleth and its underlying components.

2.5 Disaster Recovery Plan Assessment

A disaster recovery plan is a set of policies, tools, and procedures designed to restore and improve critical IT infrastructure and resources after any type of disaster, whether it's a system failure, man-made, or natural (Thiyagarajan, 2020).

For the university's infrastructure, including the authentication servers, a robust Disaster Recovery Plan (DRP) is essential to ensure availability and stability. This section evaluates the current DRP, examines its strengths and weaknesses, and highlights how automation can enhance recovery speed, consistency, and reliability.

2.5.1 Current DRP Overview

Current DRP, as documented in the department's knowledge base Dokuwiki, includes several manual processes for handling server failures and restoring critical configurations. The key scenarios addressed include:

1) Configuration File Loss or Corruption:

- If configuration files are lost or corrupted, recovery relies on Btrfs snapshots created within the past 72 hours.
- Steps include identifying the affected service, determining the snapshot to restore, and using tools like snapper and rsync to recover lost files.
- For files older than 72 hours, backups are restored using the centralized Networker backup system.

2) Master Server Outage:

- In case of a failure, the IP/network configuration is updated to redirect traffic to one of the two backup servers.
- The setup includes three authentication servers, with one acting as the main and the other two as backups. Additionally, five IP addresses are allocated for this setup to facilitate failover.
- Ensuring identical configurations and updates across all servers is critical to prevent discrepancies that could complicate disaster recovery. Uniformity minimizes recovery time and reduces the likelihood of configuration-related errors.

3) Total Server Loss:

- Recovery from total server loss involves cloning the affected server from a VMware or KVM template and restoring files from backups.
- The restoration process often requires significant manual intervention to reconfigure services and ensure operational stability.

While these steps are functional, they rely heavily on manual intervention, increasing recovery time and the risk of human error. For instance, if the backups of the servers managed by the vCenter Server Appliance for VMware were destroyed or otherwise compromised, the affected servers would need to be rebuilt and reconfigured from scratch, a time-consuming and complex process. In such cases, redeploying the identity servers from scratch using Ansible would significantly accelerate recovery and ensure consistent, identical configurations. These strategies are not unique to the identity servers, similar approaches apply to all of our servers, as both physical servers and VMs are backed up using Networker. Leveraging Ansible for rapid and uniform server deployment would minimize downtime and reduce the complexity of recovery efforts.

2.5.2 Gaps and Risks

The current DRP is comprehensive in covering potential failure scenarios, but several gaps and risks remain:

- **Manual Dependencies:** Many recovery tasks, such as selecting snapshots, reconfiguring services, or restoring databases, are performed manually. This dependency increases the likelihood of errors and delays during high-pressure recovery situations.
- **Outdated Practices:** While functional, some steps rely on older tools and methodologies that may not align with modern disaster recovery best practices.
- **Documentation Challenges:** While the Dokuwiki provides a step-by-step guide, the fragmented nature of the documentation can slow down recovery efforts, especially for new team members.

These challenges highlight the importance of incorporating automation tools such as Ansible into the DRP to eliminate manual dependencies and ensure consistency across all recovery scenarios.

Sometimes, however, it may simply be quicker and more efficient to deploy a new, identical server from scratch rather than attempt to recover fragments from multiple backup sources.

This ties directly to the next subchapter, which explores the concept of managing the server lifecycle using Ansible to streamline deployment and updates.

2.6 Server Lifecycle Management with Ansible

Efficient server lifecycle management is a key component of maintaining a robust and reliable infrastructure. The ability to seamlessly update, deploy, and replace servers is critical not only for disaster recovery but also for routine maintenance and upgrades. This approach ensures consistent configurations, reduces downtime, and enables teams to respond swiftly to changing requirements or failures.

2.6.1 Different Approaches to Server Management – PETS VS. CATTLE

Pets refer to servers that require special treatment, individual customization, and attention. They are regarded as individual entities within a server infrastructure and are often manually configured and continuously closely monitored. Upon encountering any issue or malfunction, a detailed and personalized approach is undertaken by administrators, prioritizing the swift resolution of the problem to minimize downtime. These servers are metaphorically nursed back to health (Riedel, 2021), emphasizing the inclination to maintain the server's existing state and configuration. They are considered indispensable, allowing for direct configuration changes suiting any specific needs. The management of pet servers primarily involves a manual, hands-on approach. Notably, the pet approach is oriented towards mitigating failure by investing in preventive measures and proactive maintenance strategies.

Cattle on the other hand, refer to servers that belong to a larger identical group and are not individually distinguished. They are designed without any special considerations or customizations. When encountering errors or issues, a cattle server is swiftly taken down and replaced by another one, often rebuilt from scratch. A typical use of cattle configurations would be web server arrays, search clusters, no-sql clusters, datastores and big-data clusters. (Hava Team, 2020). However, there is essentially no limitation to which servers can be managed as cattle—although some may require more effort due to dependencies on specific data sources, large data collections, or other specialized assets (DSS, personal communication, November 28, 2024). These servers are managed through automated tools and processes and follow standardized configurations.

Failures are inevitable, when dealing with infrastructure. Having an automated fast solution helps the recovery to be quick and effective. Without it, we would need plenty of time to troubleshoot the existing issue and to fix it.

Automated management plays an important role in reducing downtime and increasing the scalability withing cattle configurations. By using automated tools and processes, organizations can quickly identify and respond to server problems, mitigating the impact of failures on system performance. Not only commercial entities like Netflix (Amazon Web Services, Inc., 2016) and Airbnb (Amazon Web Services, Inc., 2019) rely on automated server management in their cloud-based infrastructure to ensure uninterrupted service delivery, but also educational institutions like Stanford University. This enables universities to promptly replace or replicate failed instances, ensuring continual access to critical resources for students, faculty, and administrative staff. Furthermore, similar to systems in commercial use, the university's infrastructure incorporates auto-scaling capabilities,

dynamically adjusting resources during peak academic periods. This automated scalability optimizes resource utilization and maintains seamless operations without manual intervention. Ultimately, automated management gives organisations the ability to maintain high availability, recover quickly from failures and adapt efficiently to changing demands, embodying the core principles of a cattle approach to server management.

2.6.2 Practical Applications in Our Environment

A key objective of the university's infrastructure is to streamline server upgrades, enabling the quick deployment of new versions with minimal manual effort. A colleague succinctly described the desired approach as: "simple server upgrades — remove and deploy a new one with the correct version." This emphasizes the need for a modern, flexible deployment process that eliminates the time-consuming and error-prone steps currently in place.

At present, major Shibboleth updates are handled manually:

- **Initial Manual Update:** One server is selected for testing and updated manually. This involves refining configurations and resolving potential issues as they arise.
- **Adjustments and Rollout:** Based on the test results, configurations are adjusted and applied to the remaining servers, leveraging predefined settings or Ansible playbooks where possible.

This process is labor-intensive and inefficient, particularly given that the university operates three production IdP servers and a test environment. Even for minor updates, where configuration changes are minimal or unnecessary, administrators must still install new versions manually across all servers.

Such manual intervention consumes valuable time, introduces inconsistencies, and increases the risk of errors. Instead of maintaining and updating individual servers, a more efficient approach would involve destroying outdated servers and deploying new, identical ones preconfigured with the latest versions of all software and dependencies using Ansible.

This shift would:

- Eliminate the need for ongoing manual configuration adjustments on each server.
- Ensure uniformity across all servers, reducing the risk of discrepancies.
- Drastically reduce the time required for both major and minor updates.

2.7 Opportunities for Automation and Improvement

The analysis of the university's IdP servers infrastructure highlights numerous opportunities for implementing automation, particularly through Ansible, to significantly enhance efficiency, consistency, and scalability. While the current setup is functional, many processes are reliant on manual intervention.

One area that stands out for improvement is disaster recovery. Currently, restoring services relies on manual processes, such as selecting and restoring snapshots, reconfiguring IP/network settings, and rebuilding databases. These steps can be automated to reduce downtime and ensure faster recovery. For instance, predefined Ansible playbooks could automate the restoration of configuration files, apply DNS changes seamlessly, and redeploy critical services with minimal intervention. In some cases, instead of relying on backups to piece together a server, it may be quicker and more efficient to deploy a new, identical server preconfigured with the latest versions of all required software. This approach not only saves time but also aligns with the cattle-style server management philosophy.

Automation can also transform server lifecycle management, simplifying both routine upgrades and major updates. Currently, the process of manually updating one server as a test and then replicating adjustments across the remaining servers is both labor-intensive and inconsistent. With Ansible, servers can be treated as disposable resources, allowing administrators to remove outdated servers and replace them with freshly deployed, preconfigured instances. This approach eliminates the need for manual configuration adjustments during updates, ensures uniformity across servers, and drastically reduces the time required for upgrades. It also simplifies the addition of new servers during peak usage periods or disaster recovery scenarios, as configurations can be standardized and deployed dynamically.

In addition, adopting Documentation as Code (DaC) practices makes the server setup more accessible and transparent for other team members, as all configuration steps are documented in code. This codified approach supports a collaborative environment, enabling other administrators to understand, modify, and deploy the configurations as needed. In summary, automating the deployment and management of the authentication servers would provide considerable gains in reliability, speed, and scalability, ultimately reducing operational costs and enhancing security. Dynamic templating can be used to manage environment-specific differences (e.g., between production and test servers), while version control ensures all configuration changes are tracked and easily reversible. These practices enable the infrastructure to remain adaptable and resilient in the face of future changes, such as updates to Shibboleth or other dependencies.

Monitoring and maintenance processes also stand to benefit from automation. For example, the installation and configuration of monitoring tools like Xymon could be fully automated, ensuring consistency across servers. Additionally, automated responses to alerts—such as restarting failed services or scaling resources dynamically—can enhance the infrastructure's ability to respond to issues proactively.

The adoption of automation and the cattle-style approach to server management represents a significant shift in how servers are deployed and maintained. Instead of treating servers as unique, customized entities that require individual care and attention, they are treated as interchangeable components of a larger system. This philosophy enables rapid replacement of failed or outdated servers, ensuring that updates, recovery, and scaling are executed efficiently.

By leveraging Ansible to automate deployment, recovery, updates, and maintenance, the university can achieve a more flexible and reliable infrastructure. These changes will not only improve the stability of critical services, such as authentication, but also reduce the workload for administrators and minimize the risks associated with manual intervention. Ultimately, automation aligns with the university's goals of creating a modern, scalable, and resilient IT environment that is prepared for both current demands and future challenges.

2.8 Summary

In this chapter, we conducted a thorough analysis of the university's current authentication server infrastructure, including its documentation, disaster recovery processes, server management practices, and configuration workflows. By reviewing existing resources, we gained insights into the server structure and configurations necessary for supporting the university's Shibboleth IdP servers. This analysis revealed several opportunities for automation, such as streamlining disaster recovery, standardizing server deployment, automating configuration management, and adopting a cattle-style server lifecycle approach. These improvements would reduce manual intervention, enhance consistency, and align the infrastructure with modern best practices, laying the groundwork for the automation plan outlined in the next chapter.

3 Plan for proposed improvements and automation

This chapter focuses on the process of designing and developing new Ansible roles for deploying fully functional Shibboleth IdP servers. It begins by presenting the planning process, which involves identifying and organizing the roles and tasks necessary to ensure that all dependencies, configurations, and components required for a Shibboleth IdP server are addressed.

3.1 Preparation based on known configuration

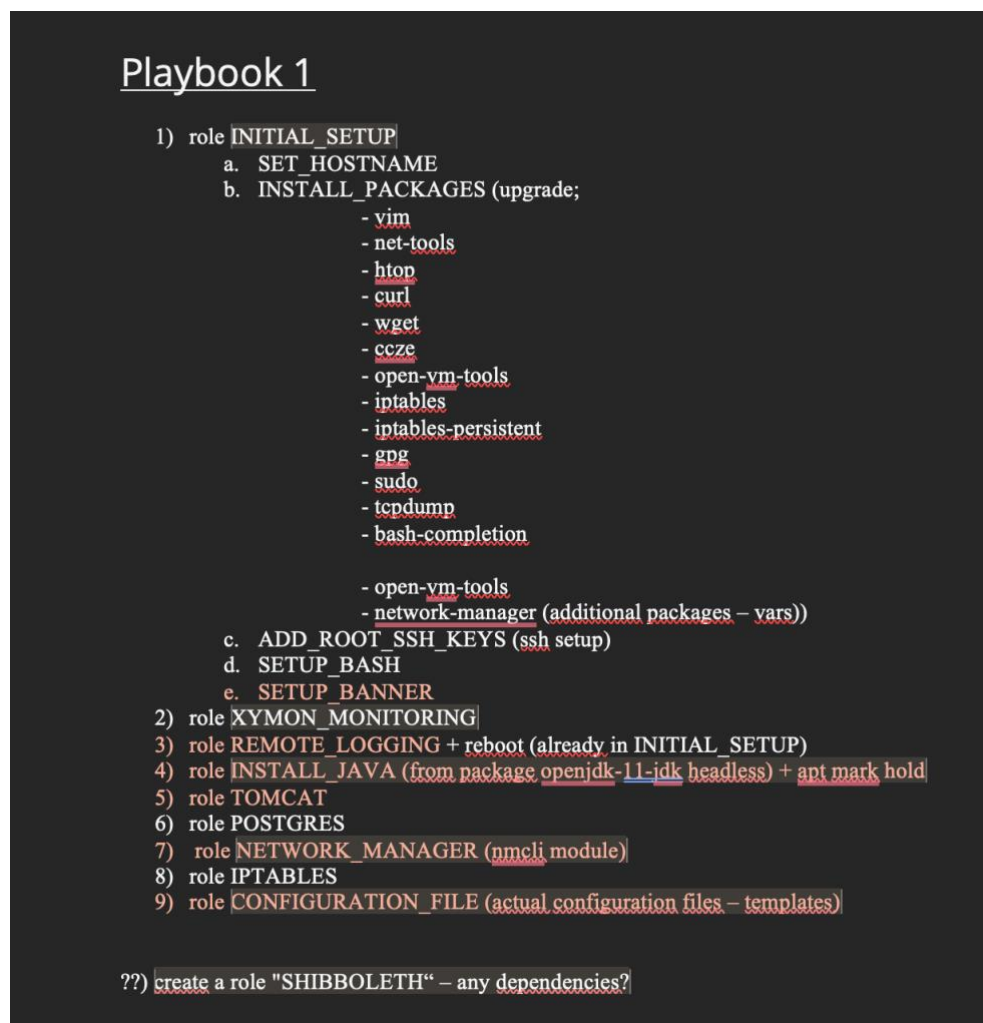


Image 1 First plan proposal for Shibboleth IdP servers playbook structure (Source: author)

As identified in the chapter 2.4.1 the Shibboleth IdP server requires several key services and system components for proper operation, including system setup, Java, Tomcat, and Shibboleth-specific configurations. Based on these findings, I created a structured list of

services and components and organized them into distinct roles, as shown in the playbook draft (see Image 1). This was the first draft of the proposed structure.

It is better to take the time to create separate roles and generalize them, even though this approach requires more time and careful planning. However, this modular design ensures reusability, allowing individual roles to be applied across other servers in the infrastructure that share similar requirements.

While playbooks are a powerful method for automating with Ansible, writing all of your tasks in a playbook isn't always the best approach. In instances where scope and variables are complex and reusability is helpful, creating most of your automation content in Ansible Roles and calling them within a playbook may be the more appropriate choice (RedHat Inc., 2024).

For this exact reason, instead of combining everything into one playbook, I took the time to separate services and software required on our servers into distinct roles. This structure provided a clear baseline, allowing me to see what belonged where and how all components would come together. From this starting point, I began creating each role, specifying what tasks needed to be completed to achieve the desired state on the target server. While the initial design served as a foundation, the roles were refined during development to meet both the specific needs of this project and the broader infrastructure requirements.

To streamline the process, I used four existing roles from the university's Ansible inventory and developed new roles for tasks such as the initial setup, Java, Tomcat, and Shibboleth configurations.

3.1.1 Initial setup role

If the admins at these companies wanted to set up a new server exactly like one that is currently running, they would need to spend a good deal of time going through all of the installed packages, documenting configurations, versions, and settings; and they would spend a lot of unnecessary time manually reinstalling, updating, and tweaking everything to get the new server to run close to how the old server did (Geerling, 2020). That is exactly what we are trying to avoid and work on.

As discussed in the earlier chapter 2.3 the initial setup process for a new server included several manual configurations. Our processes align with the focus on security and recommendations for deploying new servers.

The first configuration to be performed on any new server—especially any server with any exposure (direct or indirect) to the public Internet)—is security configuration.

There are nine basic measures to ensure servers are secure from unauthorized access or intercepted communications:

- 1. Use secure and encrypted communication.*
- 2. Disable root login and use sudo.*
- 3. Remove unused software, open only required ports.*

4. *Use the principle of least privilege.*
5. *Update the OS and installed software.*
6. *Use a properly configured firewall.*
7. *Make sure log files are populated and rotated.*
8. *Monitor logins and block suspect IP addresses.*
9. *Use SELinux (Security-Enhanced Linux).*

(Geerling, 2020)

During interviews with colleagues, it was decided to revise this approach to achieve a more clear, flexible and maintainable structure. Similarly to when the Xymon monitoring and logging tasks were separated into dedicated roles. *This approach makes the playbooks easier to understand and maintain. By isolating specific functionalities into separate roles, troubleshooting and updates can be performed more efficiently. It also provides bigger flexibility, as monitoring and logging requirements may change independently of other tasks. Separate roles allow us to update or expand these functionalities without affecting unrelated configurations* (DSS, personal communication, April 28, 2024).

The rest of the tasks from the initial setup were automated through Ansible, simplifying the deployment process significantly. The current approach follows these steps:

1) Template preparation:

SSH keys are saved on the VM template. When a new virtual machine is created from this template, the SSH keys are already in place, enabling secure access. Additionally, a script is used to modify the machine-id to avoid conflicts.

2) Manual registration of new server:

Once the VM is created, IP/network settings are configured manually in the IPAM Sauron system, followed by updating the IP address configuration on the server itself. This step remains manual because:

1. Until the IP/network settings are applied, there is no reliable network connection to the new server, making it impossible to execute Ansible playbooks remotely.
2. Automating network configuration in Sauron is not doable at this stage, as it involves administrative processes and dependencies on external systems.

3) Run Ansible role:

With SSH access and accurate network settings in place, the prepared playbook can be executed to complete the setup process. At this point, Ansible handles the remaining configurations automatically, ensuring that tasks such as firewall setup, disabling root access, and secure logging are performed consistently.

This approach not only streamlines the deployment process but also ensures that key security configurations are applied as part of the automated setup. By incorporating modular roles and adhering to security best practices, we have created a reliable and efficient baseline for deploying servers in the infrastructure.

The detailed structure and tasks executed by the `initial_setup` role, along with all other newly created roles, are documented in Appendix A.

3.1.2 Creation of roles

The process of creating roles began by identifying the commands and steps that are consistently used to achieve the desired outcomes. For each of the roles, when put together essentially, the entire Ansible code serves as documentation for the process, providing a clear guide on how to configure and set up specific components.

Before diving into the specific roles and playbooks for the Shibboleth IdP servers, we tested the `initial_setup` role on other servers that needed to be deployed in the meantime. This allowed us to observe the benefits and time saved firsthand. This role includes tasks that we had always executed manually on new servers and were already familiar with. When I wasn't sure about certain commands, I consulted colleagues to better understand their practices and approaches.

However, for roles like `java`, `tomcat` and `shibboleth` itself I truly had to expand our documentation significantly by researching how to install and configure them properly. This additional documentation ensures consistency and reliability in how these components are deployed.

While creating these roles, I looked for inspiration from publicly available ones on the internet to see if I could adapt any best practices for our needs. One role I explored was the Ansible Java role by GeerlingGuy, which is designed to install and configure Java across multiple operating systems (Geerling, 2024). We had already been using his role for setting up and configuring Apache (Geerling, 2024). Although the Java role provided some good ideas, I found it a bit too complicated for our situation. For our setup, a simpler role focused only on Debian was enough, as Java installation really just requires the download command, setting the necessary variables, and reloading the environmental variables. I decided to simplify the role to avoid the extra complexity.

Similarly, I reviewed the Ansible role for Shibboleth IdP from CESNET (Kuba, 2019). However, this role assumes the use of a standard Tomcat installation, which differs from our implementation, where we utilize two separate Tomcat user instances. This difference in approach required significant modifications to create the role for our specific needs. While this external roles offered valuable insights, I had to customize our Tomcat and Shibboleth roles to fit the university's infrastructure and requirements, creating roles that are simpler and more focused on our use-case.

3.2 Ansible Architecture

In our Ansible setup, the hosts file groups servers together to allow roles and playbooks to be executed on specific groups if needed. This grouping ensures that tasks can be executed on all relevant servers in an organized and efficient manner (for example all id authentication servers).

Since we began using Ansible, we have created several roles to automate various manual tasks that we frequently perform. Most of the servers have specific playbooks that document what is installed on the server and how it is configured. We primarily use host variables, with a focus on each host specifically and sometimes there are default variables within roles specified.

Additionally, specific files are included in the `/playbooks/files` directory and are used when a playbook runs. Some roles also require templates, which are always stored within the role. These templates allow us to dynamically modify configuration files and load them onto the target host with the correct settings, also documenting the necessary configuration details for each server.

Ansible's theory and fundamentals are well-documented in existing resources. I highly recommend the works of Olexová and Jochim, which I have referenced throughout this thesis, as they thoroughly explain the basics of Ansible (Olexová also provides a guide on how to install Ansible on Ubuntu). In addition, there are numerous online guides and literature available, such as *Ansible for DevOps* by Jeff Geerling. As these resources provide a comprehensive understanding of Ansible, I have chosen not to repeat the theoretical parts here, and this thesis focuses on the practical application instead.

3.3 Documentation as Code

Many tasks you write will be fairly obvious when you write them, but less so six months later when you are making changes. Just like application code, Ansible playbooks should be documented so you spend less time familiarizing yourself with what a particular task is supposed to do, and more time fixing problems or extending your playbooks (Geerling, 2020).

To address this, we follow the principle of naming tasks descriptively, ensuring the name clearly reflects what the task does, rather than using shortcuts or codenames. This makes it easier to understand the playbooks in the future and helps new team members onboard more quickly, as they can read a plain-language explanation alongside the code. Whenever a task, role, or playbook includes a step that might not be immediately self-explanatory, I made a point to include comments explaining its purpose and what it ensures or executes on the server.

In addition to in-line documentation, we maintain an overview of roles, playbooks, variables, handlers, and their usage in a `README.md` file for each role. This `README` provides a high-level summary, making it easier to find and understand the purpose of each role. As shown in the image below (see Image 2), the `README` structure helps us immediately identify what each role contains, what variables need to be defined or can be customized, and whether the role includes any specific tagged tasks.

Initial setup role

- Zajišťuje počáteční konfiguraci serveru, která zahrnuje: nastavení hostname a NTP serveru, aktualizaci systému, instalaci základních i volitelných balíčků, konfiguraci SSHD a další nezbytné kroky k přípravě serveru na produkční nasazení.
 - `set_hostname.yml` nastaví hostname systému podle proměnné `hostname` (definované v `playbooks/host_vars`) a aktualizuje soubor `/etc/hosts`, aby obsahoval správné hostname a FQDN (proměnná `fqdn`) pro daný server
 - `set_ntp.yml` nastaví adresu NTP serveru v souboru `timesyncd.conf` a vyvolá notifikaci (handler) pro restart služby NTP
 - `upgrade_packages.yml` provede aktualizaci všech nainstalovaných balíčků na jejich nejnovější verzi pomocí správce balíčků APT
 - `install_basic_packages.yml` instaluje základní balíčky (např. vim, curl, rsyslog, bash-completion a další) potřebné pro základní funkčnost serveru, pokud ještě nejsou nainstalovány
 - `install_additional_packages.yml` provede instalaci dalších balíčků, které jsou uvedeny v proměnné `additional_packages` (definované v `playbooks/host_vars`)
 - `setup_autoupdate_apt.yml` nastaví denní automatické aktualizace balíčků pomocí úpravy souboru `/etc/apt/apt.conf`
 - `setup_bash.yml` stáhne a aplikuje vlastní nastavení bash pro uživatele root z Git repozitáře
 - `secure_ssh.yml` nastaví bezpečnou konfiguraci pro SSH (povolí přihlášení uživatele root pouze přes SSH klíč, zakáže přihlašování heslem, zakáže X11 forwarding, zruší zobrazování MOTD), vyvolá notifikaci (handler) pro restart služby SSHD
 - (`add_root_ssh_keys.yml` - není v `main.yml`) nastaví autorizované klíče pro administrátory OSP na základě seznamu (proměnné `osp_admins`) ve `vars/main.yml` a odpovídajících souborů s klíči v adresáři `files/keys` - tento task není zahrnut v `main.yml` pro VM nasazení, ale zůstává relevantní pro fyzické servery, kde nejsou použity VM šablony
- handlers: Restart ntp service, Restart rsyslog service a Restart sshd service

Image 2 Documentation of Initial setup role on GitLab (Source: author)

The documentation is always pushed to GitLab along with the code itself (see Code 1).

Code 1 Documentation of Initial setup role written as code (Source: author)

```
57 ## Initial setup role
58 - Zajišťuje počáteční konfiguraci serveru, která zahrnuje: nastavení hostname a NTP serveru, aktualizaci systému, instalaci základních i volitelných
balíčků, konfiguraci SSHD a další nezbytné kroky k přípravě serveru na produkční nasazení. \
59 ``set_hostname.yml`` nastaví hostname systému podle <span style="color:#8bd6f2"> proměnné </span> hostname (definované v playbooks/host_vars) a
aktualizuje soubor /etc/hosts, aby obsahoval správné hostname a FQDN (<span style="color:#8bd6f2"> proměnná </span> fqdn) pro daný server \
60 ``set_ntp.yml`` nastaví adresu NTP serveru v souboru timesyncd.conf a vyvolá notifikaci (handler) pro restart služby NTP \
61 ``upgrade_packages.yml`` provede aktualizaci všech nainstalovaných balíčků na jejich nejnovější verzi pomocí správce balíčků APT \
62 ``install_basic_packages.yml`` instaluje základní balíčky (např. vim, curl, rsyslog, bash-completion a další) potřebné pro základní funkčnost serveru,
pokud ještě nejsou nainstalovány \
63 ``install_additional_packages.yml`` provede instalaci dalších balíčků, které jsou uvedeny v <span style="color:#8bd6f2"> proměnné </span>
additional_packages (definované v playbooks/host_vars) \
64 ``setup_autoupdate_apt.yml`` nastaví denní automatické aktualizace balíčků pomocí úpravy souboru /etc/apt/apt.conf \
65 ``setup_bash.yml`` stáhne a aplikuje vlastní nastavení bash pro uživatele root z Git repozitáře \
66 ``secure_ssh.yml`` nastaví bezpečnou konfiguraci pro SSH (povolí přihlášení uživatele root pouze přes SSH klíč, zakáže přihlašování heslem, zakáže X11
forwarding, zruší zobrazování MOTD), vyvolá notifikaci (handler) pro restart služby SSHD \
67 (``add_root_ssh_keys.yml`` - není v main.yml) nastaví autorizované klíče pro administrátory OSP na základě seznamu (<span style="color:#8bd6f2">
proměnné </span> osp_admins) ve vars/main.yml a odpovídajících souborů s klíči v adresáři files/keys - tento task není zahrnut v main.yml pro VM
nasazení, ale zůstává relevantní pro fyzické servery, kde nejsou použity VM šablony
68 - handlers: Restart ntp service, Restart rsyslog service a Restart sshd service
69
```

GitLab is used for version control and collaboration, with the entire Ansible code stored in our GitLab repository. We leverage features such as separate branches, merge requests, and code reviews to ensure proper collaboration and maintain high quality control. This workflow enables us to track changes, preserve a clear history of modifications, and ensure that only thoroughly reviewed code is deployed. Team members can work independently on their tasks in isolated branches, and before merging changes into the main branch, the code undergoes reviews to maintain consistency, avoid errors, and improve the overall quality of our infrastructure automation.

Previously, we used DokuWiki as our primary documentation system, which served as a source of reference for many of the decisions made in this project (chapter 2.2). However, as discussed earlier, DokuWiki presented several challenges. *When a page in the documentation is outdated, it can confuse new employees, remote workers, or end users, impacting productivity. If you keep documentation in a separate system, development teams often forget about it – in the worst case, there is no clear source of truth for documentation* (Swimm Team, n.d.).

3.4 Configuration Management and Role Standardization

A core benefit of Ansible is its ability to enforce consistent configurations across servers, reducing manual intervention and eliminating discrepancies. Configuration management ensures that all servers, whether Shibboleth IdP or others, are maintained in a standardized and predictable state.

The roles developed for the Shibboleth IdP server in this work are designed to be modular and reusable. By using well-structured variables, templates, handlers, and tasks, these roles can be adapted for various servers in the university's infrastructure. Variables allow environment-specific adjustments without requiring changes to the underlying role logic. Templates provide dynamically generated configuration files, while handlers ensure that services are restarted or reloaded only when necessary, reducing unnecessary downtime.

Ansible uses Jinja2 templating to enable dynamic expressions and access to variables and facts as is stated in the documentation. You can create a template for a configuration file, then deploy that configuration file to multiple environments and supply the correct data (IP address, hostname, version) for each environment (Ansible project contributors, 2024). For an example of such template see Code 2.

Code 2 Example of using variables within Jinja2 template (Source: author)

```
29  ## DN resolution properties ##
30
31  # Search DN resolution, used by anonSearchAuthenticator, bindSearchAuthenticator
32  # for AD: CN=Users,DC=example,DC=org
33  idp.authn.LDAP.baseDN           = {{ ldap_base_dn }}
34  idp.authn.LDAP.baseDNGroups     = {{ ldap_base_dn_groups }}
35
36  idp.authn.LDAP.subtreeSearch    = true
```

Additionally, the use of tags for specific setups allows for precise execution of tasks, enabling more granular control over which parts of the playbook are executed based on the server's needs or environment. This combination ensures that roles are not only modular but also adaptable, enabling efficient deployment and management across various infrastructures without redundancy.

Standardizing roles has multiple advantages. It improves maintainability, as changes can be made centrally and applied universally. It also accelerates deployment, as predefined roles eliminate the need to write custom scripts or configurations for each new server. Moreover, version control systems, such as Git, can track role changes, enabling easy rollback in case of errors.

To maximize efficiency, this thesis proposes maintaining a centralized repository of Ansible roles, organized and documented for team-wide accessibility. This repository will act as a single source of truth for configurations, ensuring alignment across environments.

3.5 Summary

In this chapter, the plan for proposed improvements and automation through Ansible was presented. As the third objective, four new Ansible roles were developed and documented: `initial_setup`, `java`, `tomcat`, and `shibboleth`. These roles are included in the Appendix A and will be integrated into a playbook with existing roles used by the DSS team in the next chapter, as it was outlined in the plan draft.

4 Implementation

This chapter details the practical execution of the automation plan from previous chapter, illustrating the steps taken to deploy, configure, and manage Shibboleth IdP servers using Ansible. It also highlights the efficiency gains and future potential for the implemented solution.

4.1 Playbook

The central component of the automation process is the Ansible playbook, which orchestrates the deployment and management tasks required for the Shibboleth IdP servers. This playbook was built by combining designed roles with additional tasks introduced to address ad hoc needs, ensuring flexibility and comprehensive functionality during Shibboleth version upgrade. Its modularity allows it to handle both routine processes and evolving requirements efficiently.

The foundation of the playbook was established using Ansible roles, which were created during the planning phase to automate repetitive and critical tasks. They are combined with the roles that are already in use in DSS. While these roles address most routine needs, the implementation process revealed specific requirements that could not be fully anticipated. For these, the playbook includes ad hoc tasks added as standalone actions or extensions to existing roles (as presented in Code 3). For example, during testing or emergency scenarios, temporary configuration adjustments were needed to address short-term requirements.

Code 3 Playbook for id servers (Source: author)

```
playbooks > Ⓜ id4.yml
1  ---
2  - hosts: id4
3    roles:
4      - initial_setup
5      - iptables
6      - postgres
7      - java
8      - apache
9      - tomcat
10     - shibboleth
11     - xymon_monitoring
12     - remote_logging
13     - networker_backup
14
15   tasks:
16     - name: Copy apache configuration for id4
17       ansible.builtin.template:
18         src: templates/id4/id.vse.cz.conf
19         dest: /etc/apache2/sites-available/id.vse.cz.conf
20         owner: root
21         group: root
22         mode: '0640'
23
24     - name: Create a symlink in sites-available for new configuration
25       file:
26         src: /etc/apache2/sites-available/id.vse.cz.conf
27         dest: /etc/apache2/sites-enabled/id.vse.cz.conf
28         state: link
29
30     - name: Remove file sites-available/default-ssl.conf
31       ansible.builtin.file:
32         path: /etc/apache2/sites-available/default-ssl.conf
33         state: absent
34
35     - name: Remove file sites-available/vhosts.conf
36       ansible.builtin.file:
37         path: /etc/apache2/sites-available/vhosts.conf
38         state: absent
39
40     - name: Remove file sites-enabled/000-default.conf
41       ansible.builtin.file:
42         path: /etc/apache2/sites-enabled/000-default.conf
43         state: absent
44
45     - name: Remove file sites-enabled/vhosts.conf
46       ansible.builtin.file:
47         path: /etc/apache2/sites-enabled/vhosts.conf
48         state: absent
49
```

4.1.1 Ansible vault

Ansible Vault is a powerful feature used to securely manage sensitive information, such as passwords, API keys, and private keys, within Ansible playbooks. By encrypting sensitive data, it ensures that credentials and other confidential information are not exposed in plain text, even if the playbook or role files are shared or stored in version control systems like Git. In this project, Ansible Vault was utilized to protect critical credentials, including those for accessing the Shibboleth IdP servers and managing SSL certificates. Below is an example from `host_vars` file for id servers, where ansible vault is used (see Code 4). This approach not only enhances security but also simplifies compliance with data protection policies, enabling safe and efficient automation without compromising sensitive information.

Code 4 Section of host_vars file for id servers (Source: author)

```
20  users:
21      - name: shibboleth
22        password: !vault |
23            $ANSIBLE_VAULT;1.1;AES256
24            6637306435386233364636464613639633833626338656562
25            6133373138316635616537383331343037333363383561640a
26            32303534643434653736613461373563663231346636393163
27            6262386636333665660a6664336462626165303532376165
28            37653739623837376535613765383031646232623035333038
29            66373933363131323037373530643430373363396465396536
30  db_access_host: 127.0.0.1
31  java_package: openjdk-17-jdk
32  java_home: java-17-openjdk-amd64
```

4.2 Testing and Validation

Testing and validation are critical for ensuring that the automation workflows developed in this thesis are reliable and error-free. Ansible's built-in testing tools, such as Ansible Lint, can be used to validate roles, playbooks, and configurations before running them on the server.

Ansible Lint is a popular tool for ensuring best practices and code quality by identifying potential issues in playbooks and roles before they are applied in production. It helps maintain a clean and errorless codebase by flagging common mistakes and enforcing coding standards.

The testing process involved several stages:

- 1) Role Testing: Each Ansible role was tested in isolation to ensure that it performs its intended function. For example, the role for configuring Shibboleth IdP was tested to verify that it generates valid configuration files and restarts the required services correctly.
- 2) Integration Testing: The playbook combining multiple roles was tested in a controlled environment to verify that all components work together seamlessly. This is particularly important for complex workflows, such as disaster recovery or server deployment.
- 3) Environment Testing: Testing was performed on non-production environment that replicates the production setup. This ensures that updates, patches, or new configurations do not disrupt live services.

The greatest advantage of using Ansible lies in its iterative nature: you can write a role, run it, and, if a task fails, refine it until it works. Ansible is particularly effective for debugging because it provides clear, detailed error messages, often pinpointing the issue directly. Another key benefit is how efficiently you can recover from mistakes. For example, if your configuration breaks something and you're unsure how to fix it, you can simply return to the latest working state and adjust the following tasks until the desired state is achieved. This process is both quick and effective, unlike manual deployment where fixing issues would typically involve either repeating the entire process (which is time-consuming) or maintaining constant backups (e.g., snapshots).

4.3 Time Efficiency and Productivity Gains

The introduction of automation through Ansible has led to significant improvements in time efficiency for server deployment and management tasks. I measured how long it takes to complete these tasks manually for all processes except the Shibboleth configuration. The time for Shibboleth configuration is only a rough estimate, but I believe it would have taken me much longer, and I would likely have gotten lost in the configuration files. I then measured the time it took for the corresponding Ansible roles to execute each task and compared these measurements. The first graph (Figure 2) illustrates the time comparison between manual execution and automated roles for various tasks, grouped under defined roles, measured in minutes. Tasks that previously required tens of minutes can now be completed in just a few minutes using Ansible playbooks. This represents a dramatic reduction in time, with some tasks achieving up to 90-95% time savings.

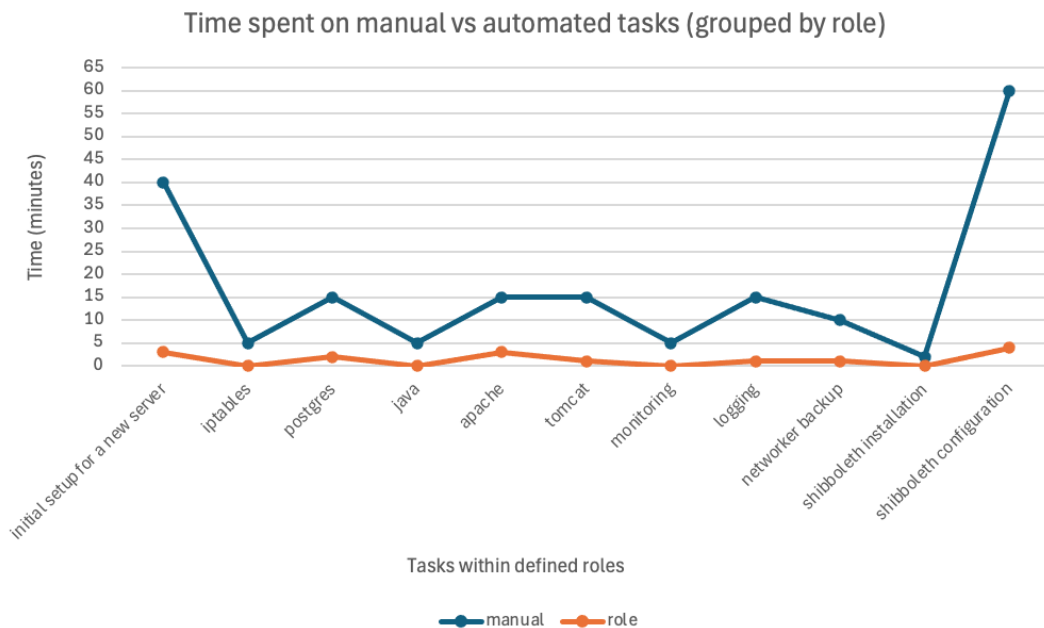


Figure 2 Time spent on manual vs automated tasks (Source: author)

For example, deploying a new Shibboleth server manually required approximately 180 minutes, as shown in the second graph (Figure 3). Through automation, this process has been reduced to under 20 minutes - an almost 90% reduction in deployment time. The ability to save time on repetitive tasks enables more focus on learning and testing new configurations during updates, which can then be incorporated into the automation roles for future deployments.

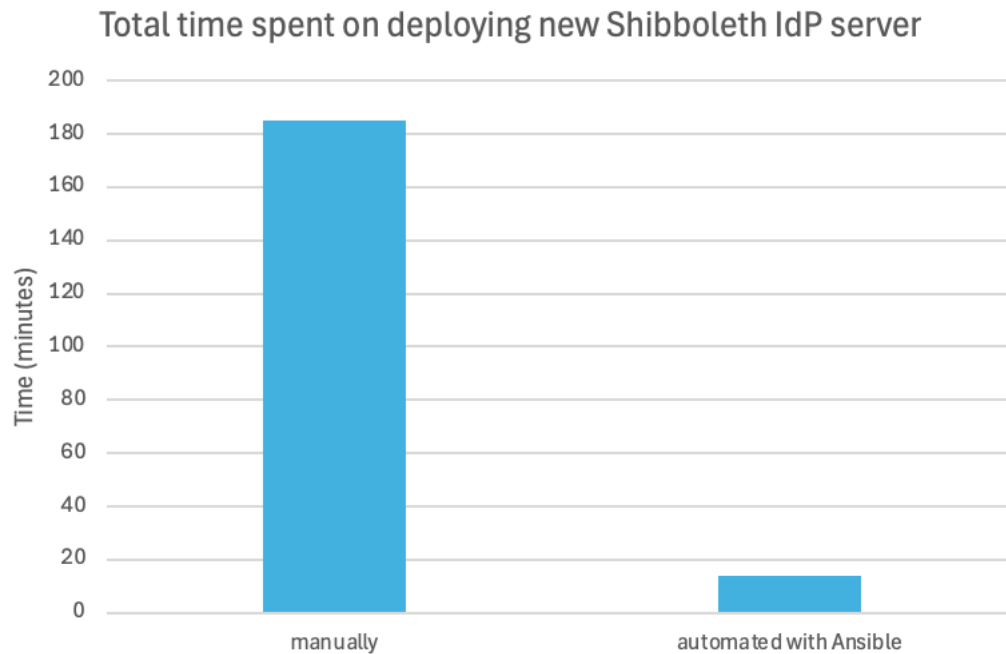


Figure 3 Total time spent on deploying new Shibboleth IdP server (Source: author)

Moreover, automating complex configurations, such as Shibboleth, has proven extremely valuable. The process of manually fine-tuning Shibboleth configurations often involved trial and error, especially when resolving errors between Shibboleth and Tomcat. Automating this setup allowed the deployment process to be repeated in minutes, enabling faster debugging cycles without needing to redo earlier tasks manually. This ensures consistency and eliminates the risk of human error in tasks that have already been successfully automated.

The adoption of automation has transformed server deployment and configuration from a time-consuming, error-prone process into a streamlined and reliable workflow. This efficiency not only accelerates the deployment process but also enhances productivity by enabling the reuse of optimized configurations across multiple servers.

4.4 Updates and Security Patch Management

Keeping servers updated with the latest software versions and security patches is essential for maintaining a secure, stable, and compliant infrastructure. Leveraging Ansible's automation capabilities simplifies this process, ensuring consistency and minimizing the risk of errors during updates.

For minor updates, the Ansible roles developed as part of this work can seamlessly apply patches, validate installations, and confirm the success of updates across all servers. By using Ansible's apt modules, updates can either be scheduled regularly or triggered on demand, significantly reducing manual effort. This is especially beneficial for addressing critical security vulnerabilities, as patches can be applied uniformly across the entire infrastructure within minutes, minimizing exposure to potential threats.

For major updates, such as upgrading the Shibboleth IdP to a newer version, the process benefits from a controlled approach enabled by automation. Instead of performing in-place upgrades, a new server instance with the updated version is deployed using Ansible, following the cattle-style server management philosophy. This method ensures all configurations, dependencies, and integrations are properly tested and applied in a predictable and repeatable manner, reducing downtime and risks associated with manual upgrades.

Additionally, automating the update process ensures compliance with security policies by maintaining a consistent patch level across the environment. It significantly lowers the risk of security breaches due to outdated software while saving valuable time and resources for IT administrators. This approach reinforces a proactive strategy for managing updates and aligns with modern best practices for infrastructure management.

4.5 Disaster Recovery Automation

The current DR strategy relies heavily on snapshots, backups, and manual intervention. By incorporating Ansible's automation capabilities, these processes can be streamlined, reducing recovery time and minimizing the potential for human error.

Below, we revisit the main disaster scenarios defined in the analysis and explore how Ansible could enhance recovery efforts.

1) Configuration file loss or corruption.

Using Ansible templates, critical configuration files can be dynamically maintained and deployed. In case of corruption or loss, a simple playbook can restore all configuration files to their desired state from a version-controlled repository. This completely replaces the Btrfs snapshots that are no longer needed.

2) Master server outage.

IP address is manually reconfigured to redirect traffic to one of the two backup servers in the event of a master server failure.

Maintaining identical configurations and updates across the master and backup servers is critical to ensure smooth failover. This process currently relies on careful manual updates and checks.

3) Total server loss.

Using Ansible roles and playbooks, a new server instance can be automatically deployed from scratch. This includes installing the required software, configuring services, and restoring settings.

For example, automating configuration recovery and failover processes can reduce reliance on snapshots and backups, simplifying disaster recovery and focusing on rapid, reliable service restoration. This aligns with modern infrastructure management principles, where automation is key to improving resilience and minimizing recovery time.

In summary, while the current DR processes are effective, incorporating Ansible into the disaster recovery strategy will further enhance speed, reliability, and consistency. By automating key recovery tasks, IT administrators can focus on strategic efforts rather than time-consuming manual procedures, ultimately improving the organization's ability to handle and recover from disasters.

4.6 Future Development and Scalability Considerations

As the university's infrastructure continues to grow, the scalability and adaptability of its automation framework will become increasingly important.

Future development will focus on expanding the use of automation to cover all aspects of server management, including:

- Converting all servers to use documentation as code.
- Automating server certificate management.
- Addressing other repetitive tasks that have not yet been automated.

Scalability also involves extending automation beyond the current infrastructure. As the university adopts new technologies, such as containerization or cloud-based platforms, the automation framework should evolve to support these changes. For example, Ansible can be integrated with Kubernetes to manage containerized applications or used alongside cloud provisioning tools to automate virtual machine deployment in public or private clouds.

Ultimately, the goal is to create a flexible, future-proof infrastructure that can adapt to changing needs without requiring significant manual effort. By continuing to invest in automation and modular design, the university can ensure that its IT environment remains efficient, reliable, and prepared for future challenges.

Additionally, a future objective is to publish newly created Ansible roles as part of an open-source Git repository. This requires conducting a detailed evaluation to ensure no sensitive data is included, making the roles safe for public use.

4.7 Summary

This chapter provides a comprehensive overview of the implementation process for automating the deployment, configuration, and management of Shibboleth IdP servers using Ansible. It showcases the development of the playbook, testing and validation efforts,

efficiency gains, update management, disaster recovery enhancements, and considerations for future scalability.

The Ansible playbook forms the core of the automation, combining modular roles with ad hoc tasks to address both routine and specific needs. Sensitive information was securely managed using Ansible Vault, ensuring confidentiality and compliance with data protection policies. Testing and validation processes, including role, integration, and environment testing, ensured reliability and minimized errors.

The automation resulted in significant time savings, reducing server deployment from hours to minutes, while ensuring consistent configurations and eliminating manual errors. Updates and patch management were streamlined, enabling rapid application of security patches and controlled upgrades with minimal downtime. Disaster recovery processes were also enhanced, with Ansible enabling faster recovery from configuration loss, server outages, or total server loss. The chapter concludes with suggestions for future developments to further enhance automation and efficiency.

Conclusions

The main goal of this bachelor's thesis was to modernize the infrastructure of Shibboleth IdP servers at VŠE using Ansible automation. This goal was successfully achieved, as demonstrated in chapter 4, which describes the implementation and the impact of automation on server management efficiency. Ansible significantly reduced the time required for server deployment, minimized the risk of human error, and enhanced disaster recovery processes.

The main goal was accomplished by meeting three secondary objectives. The first secondary objective was to document the current state of the Shibboleth servers at VŠE, including an analysis of the internal documentation and manual processes involved in their management, followed by unstructured interviews with colleagues from DSS CI. This objective was achieved in chapter 2, where the existing procedures were described and detailed analysis was conducted. Also, the second secondary objective was met as areas for improvement through automation were identified in that chapter.

The third secondary objective was to create Ansible roles and tasks focused on automating the deployment and management of authentication servers. This was fulfilled in chapter 3, where the design of roles such as `initial_setup`, `java`, `tomcat`, and `shibboleth` was introduced. Detailed documentation of these roles can be found in Appendix A. They were created with generalization in mind to ensure their applicability to other servers.

During the implementation phase, the roles were integrated into a playbook, which was repeatedly tested and validated until it was error-free and complete. This process, described in chapter 4, confirmed the functionality of the automated solution. Evaluation of automation benefits, measured by task completion times as a key metric, showed that Ansible accelerated some processes by up to 95%. Additionally, it ensured consistent server configurations and eliminated risks associated with manual setup.

Chapter 4.6 outlined possible future opportunities for further automatization of other tasks and servers and fully implementing the concept of documentation as code.

Overall, this bachelor's thesis demonstrated that automation with Ansible can bring substantial improvements not only to the management of authentication servers at VŠE but also to IT infrastructure in general. The implementation of Ansible has laid a solid foundation for further advancements and offers a transferable solution that could benefit other organizations with similar needs.

List of references

Campi, N., & Bauer, K. (2009). *Automating Linux and Unix system administration* (2nd ed). Apress.

Ghalib, U. (2021, June 30). *What is Server Automation? | Tool & Advantages*. AttuneOps. <https://attuneops.io/what-is-server-automation/>

Davis, J. (2022). *Modern system administration: Managing reliable and sustainable systems*. O'Reilly Media Inc.

Phoeteau, N. (2024). *Automating Infrastructure with Terraform and Ansible*. Independently published.

Olexová, A. (2020). *Využití platformy Ansible pro automatizaci IT infrastruktury* [Master's thesis, Prague University of Economics and Business]. InSIS. https://insis.vse.cz/zp/portal_zp.pl?prehled=vyhledavani;podrobnosti_zp=71414;zp=71414;download_prace=1

Jochim, V. (2023). *Grafické uživatelské rozhraní pro správu konfigurační systémů Ansible* [Bachelor's thesis, Prague University of Economics and Business]. InSIS. <https://insis.vse.cz/zp/82441>

JetBrains. (2023). *Infrastructure provisioning tools usage 2023*. [Graph]. Statista. <https://www-statista-com.zdroje.vse.cz/statistics/1450935/infrastructure-provisioning-tools-usage/>

Alsadi, M. (2017, June 7). *New level of automation with Ansible*. Red Hat Developer. <https://developers.redhat.com/blog/2017/06/07/new-level-of-automation-with-ansible#why-is-bash-scripting-not-automation>

Klíč, J. (2018). *Automatizace pomocí Ansible ve Windows* [Bachelor's thesis, Brno University of Technology]. https://www.vut.cz/www_base/zav_prace_soubor_verejne.php?file_id=181000

Hudec, M. (2016). *Automatizace správy síťových prvků pomocí Ansible* [Bachelor's thesis, Czech Technical University in Prague]. DSpace. https://dspace.cvut.cz/bitstream/handle/10467/64680/F3-BP-2016-Hudec-Miroslav-BP_Miroslav_Hudec.pdf?sequence=1&isAllowed=y

Johari, A. (2024, September 16). *Chef vs Puppet vs Ansible vs Saltstack: Which Works Best For You?* Edureka. <https://www.edureka.co/blog/chef-vs-puppet-vs-ansible-vs-saltstack/>

Bielik, E. (2020). *Deployment a správa aplikací v Cloude pomocí Ansible a Terraformu* [Bachelor's thesis, Prague University of Economics and Business]. InSIS. https://insis.vse.cz/auth/lide/clovek.pl?zalozka=7;id=128684;studium=178386;zp=68805;download_prace=1

GitHub, Inc. (n.d.). *ansible/ansible* [GitHub repository]. Retrieved September 18, 2024, from <https://github.com/ansible/ansible>

Pleskač, V. (2019). *Analýza testovacího prostředí Shibboleth IdP pro potřeby CI VŠE* [Bachelor's thesis, Prague University of Economics and Business]. InSIS.

https://insis.vse.cz/auth/lide/clovek.pl?zalozka=7;id=125911;studium=172844;zp=68817;download_prace=1

Department of System Support, Informatics Centre, Prague University of Economics and Business. (n.d.). *Internal DokuWiki documentation: Server id.vse.cz*. Accessed on September 20, 2024.

NAKIVO Team. (2023, June 1). *VMware VM Templates A to Z*. NAKIVO. <https://www.nakivo.com/blog/vm-templates-a-to-z/>

Cantor, S. (2024, March 21). *System requirements—Identity Provider 5—Confluence*. Shibboleth documentation. Retrieved March 28, 2024, from <https://shibboleth.atlassian.net/wiki/spaces/IDP5/pages/3199511079/SystemRequirements>

McKenzie, C. (2019, November 26). *Tomcat vs. Jetty: How these Java application servers compare and differ*. TheServerSide. Retrieved December 4, 2023, from <https://www.theserverside.com/video/Tomcat-vs-Jetty-How-these-application-servers-compare>

Zeller, T. (2024, May 30). *Tomcat 10.1*. Shibboleth documentation. Retrieved December 7, 2024, from <https://shibboleth.atlassian.net/wiki/spaces/IDP5/pages/3199511079/Tomcat+10.1>

Thiyagarajan, R. B. (2020). *Single and Multi-Cloud Disaster Recovery Management using Terraform and Ansible* [Master's thesis, National College of Ireland]. NORMA eResearch @NCI Library. <https://norma.ncirl.ie/4144/1/rambaraththiyagarajan.pdf>

Riedel, D. (2021, September 22). *Pets vs Cattle 2.0: An Analogy and Approach for Modern Infrastructures*. ERP Today. <https://erp.today/pets-vs-cattle-2-0-an-analogy-and-approach-for-modern-infrastructures/>

Hava Team. (2020, June 9). *Cattle vs Pets—DevOps Explained*. Hava. <https://www.hava.io/blog/cattle-vs-pets-devops-explained>

Amazon Web Services, Inc. (2016). *Netflix case study*. AWS. <https://aws.amazon.com/solutions/case-studies/netflix-case-study/>

Amazon Web Services, Inc. (2019). *Airbnb Case Study*. AWS. <https://aws.amazon.com/solutions/case-studies/airbnb-case-study/>

RedHat, Inc. (2024, January 8). *What is an Ansible Role—and how is it used?* Ansible Documentation. Retrieved October 29, 2024, from <https://www.redhat.com/en/topics/automation/what-is-an-ansible-role/>

Geerling, J. (2020). *Ansible for DevOps: Server and configuration management for humans* (2nd edition). Leanpub. <https://www.ansiblefordevops.com>

Geerling, J. (2024). *ansible-role-java* [GitHub repository]. GitHub. <https://github.com/geerlingguy/ansible-role-java>

Geerling, J. (2024). *ansible-role-apache* [GitHub repository]. GitHub. <https://github.com/geerlingguy/ansible-role-java>

Kuba, M. (2019). *ansible-role-shibboleth-idp* [GitHub repository]. GitHub. <https://github.com/CESNET/ansible-role-shibboleth-idp/>

Swimm Team. (n.d.). *Documentation as code: Why you need it and how to get started*. Swimm. Retrieved December 9, 2024, from <https://swimm.io/learn/code-documentation/documentation-as-code-why-you-need-it-and-how-to-get-started>

Ansible project contributors. (2024, December 3). *Templating (Jinja2)*. Ansible Community Documentation. https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_templating.html

OpenAI. (2024). Grammar and writing assistance provided by ChatGPT. Retrieved from <https://www.openai.com/chatgpt>

Appendices

Appendix A: Ansible Role Documentation

One of the partial outcomes of this project is the creation of standardized roles for services utilized in the authentication server installation process. These roles are designed to be reusable in various deployment scenarios, ensuring consistency and efficiency. This appendix provides detailed documentation for each role, including an explanation of its functionality, required variables, and the actions necessary to implement the role successfully.

ROLE: Initial setup

The `initial_setup` role serves as a foundational configuration for setting up a new server. It performs a series of preparatory tasks, ensuring the system is equipped with essential configurations, security measures, and basic packages required for further deployments. This role is used for every new server in the infrastructure and can also be reused in other deployments due to its modular structure.

The `initial_setup` role was designed based on the requirements analyzed in the chapter 2.3. Its modular structure, with separate task files, allows for flexibility and reusability.

Structure

The `initial_setup` role is located in the `ansible/roles/initial_setup` directory and consists of the following key components:

- **Tasks:** Eight separate task files, each handling a specific system configuration.
- **Handlers:** Defined actions to restart services when required. Handlers in use are Restart ntp service, Restart rsyslog service and Restart sshd service
- **Variables:** Used for modularity and adaptability of the role. Variables in the `initial_setup` role are `hostname`, `fqdn` and `additional_packages`.

Code 5 Structure of `initial_setup` role (Source: author)

```

  ▾ initial_setup
    ▾ handlers
      main.yml
    ▾ tasks
      ! add_root_ssh_keys.yml
      ! install_additional_packages.yml
      ! install_basic_packages.yml
      main.yml
      ! secure_ssh.yml
      ! set_hostname.yml
      ! set_ntp.yml
      ! setup_autoupdate_apt.yml
      ! setup_bash.yml
      ! upgrade_packages.yml
    > vars
```


Code 6 Main.yml file in initial_setup role (Source: author)

```
roles > initial_setup > tasks > main.yml
1  ---
2  - name: Set a hostname
3    ansible.builtin.import_tasks: set_hostname.yml
4
5  - name: Update NTP settings
6    ansible.builtin.import_tasks: set_ntp.yml
7
8  - name: Upgrade all packages
9    ansible.builtin.import_tasks: upgrade_packages.yml
10
11 - name: Install basic packages
12   ansible.builtin.import_tasks: install_basic_packages.yml
13
14 - name: Install additional packages
15   ansible.builtin.include_tasks: install_additional_packages.yml
16
17 - name: Ensure daily auto apt updates
18   ansible.builtin.import_tasks: setup_autoupdate_apt.yml
19
20 - name: Setup bash for the root user
21   ansible.builtin.import_tasks: setup_bash.yml
22
23 - name: Secure SSH server
24   ansible.builtin.import_tasks: secure_ssh.yml
25
```

Task files

1) set_hostname.yml

Code 7 set_hostname.yml (Source: author)

```
roles > initial_setup > tasks > ! set_hostname.yml > ...
Ansible Tasks Schema - Ansible tasks file (ansible.json)
1  ---
2  - name: Set a hostname in system
3    ansible.builtin.hostname:
4      name: "{{ hostname }}"
5
6  - name: Set a hostname in /etc/hosts
7    ansible.builtin.lineinfile:
8      dest: /etc/hosts
9      regexp: "127.0.1.1"
10     line: "127.0.1.1 {{ hostname }} {{ fqdn }}"
11     state: present
12
```

This task configures the server's hostname and updates the /etc/hosts file to ensure the hostname and fully qualified domain name (FQDN) can be resolved locally. This is

crucial for proper functionality of applications and system services that rely on hostname resolution.

Key features:

- Sets a hostname in the system using `ansible.builtin.hostname`
- Updates `/etc/hosts` with a line that maps 127.0.1.1 (the loopback address) to the hostname and fqdn using the `ansible.builtin.lineinfile` module.

2) set_ntp.yml

Code 8 set_ntp.yml (Source: author)

```
roles > initial_setup > tasks > ! set_ntp.yml > ...
Ansible Tasks Schema - Ansible tasks file (ansible.json)
1  ---
2  - name: Set address for the NTP server
3    ansible.builtin.lineinfile:
4      path: /etc/systemd/timesyncd.conf
5      regexp: '^NTP='
6      line: NTP=ntp.vse.cz
7      notify: Restart ntp service
8
```

Ensures time synchronization on the server by configuring the address of the IC's NTP server (`ntp.vse.cz`). Accurate time synchronization is critical for distributed systems, logging, and security protocols.

Key features:

- Modifies the `/etc/systemd/timesyncd.conf` file to set the NTP server address using `ansible.builtin.lineinfile`.
- Notifies the `Restart ntp service` handler to apply the new time synchronization settings.

3) upgrade_packages.yml

Code 9 upgrade_packages.yml (Source: author)

```
roles > initial_setup > tasks > ! upgrade_packages.yml > ...
Ansible Tasks Schema - Ansible tasks file (ansible.json)
1  ---
2  - name: Update package list
3    ansible.builtin.apt:
4      update_cache: true
5
6  - name: Upgrade all packages
7    ansible.builtin.apt:
8      upgrade: dist
9      lock_timeout: 320
10
```

Keeps the system up-to-date by upgrading all installed packages to their latest versions, which ensures the server starts with the latest security patches and features, reducing vulnerabilities.

Key features:

- Updates the package list and then upgrades all packages to their latest version using the `ansible.builtin.apt` module.

4) `install_basic_packages.yml`

Code 10 `install_basic_packages.yml` (Source: author)

```
roles > initial_setup > tasks > ! install_basic_packages.yml > ...
Ansible Tasks Schema - Ansible tasks file (ansible.json)
1  ---
2  - name: Install basic packages
3    ansible.builtin.apt:
4      name:
5        - vim
6        - net-tools
7        - htop
8        - curl
9        - wget
10       - ccze
11       - open-vm-tools
12       - iptables
13       - iptables-persistent
14       - gpg
15       - sudo
16       - tcpdump
17       - bash-completion
18       - rsync
19       - git
20       - rsyslog
21       - tmux
22       - ghostscript
23     state: present
24
```

Installs a set of basic utilities and tools required for server functionality. This is the list we decided to install on all our new servers within Department of System Support.

Key features:

- Includes tools such as `vim`, `curl`, `rsyslog`, `bash-completion`, and others.
- Ensures these packages are present on the system by the `ansible.builtin.apt` module.

5) install_additional_packages.yml

Code 11 install_additional_packages.yml (Source: author)

```
roles > initial_setup > tasks > ! install_additional_packages.yml > ...
Ansible Tasks Schema - Ansible tasks file (ansible.json)
1  ---
2  - name: Install additional packages
3    ansible.builtin.apt:
4      name: "{{ additional_packages }}"
5      state: present
6      when: additional_packages is defined
7
```

Allows for the installation of extra packages defined by the `additional_packages` variable. Provides flexibility to customize server installations based on specific needs, because different software is required for various applications or environments.

Key features:

- Executes only if the `additional_packages` variable is defined, providing flexibility for specific deployments.
- Uses the `ansible.builtin.apt` module to install the specified packages.

6) setup_autoupdate_apt.yml

Code 12 setup_autoupdate_apt.yml (Source: author)

```
roles > initial_setup > tasks > ! setup_autoupdate_apt.yml > ...
Ansible Tasks Schema - Ansible tasks file (ansible.json)
1  ---
2  - name: Setup apt for automatic updates
3    ansible.builtin.lineinfile:
4      path: /etc/apt/apt.conf
5      regexp: 'APT::Periodic::Update-Package-Lists 1;'
6      line: 'APT::Periodic::Update-Package-Lists 1;'
7      create: true
8
```

Configures automatic updates for the APT package manager. It basically runs `apt update` for you daily. The task is suitable if you don't want full automation of updates for example in production environments where updates need to be reviewed. By ensuring that package lists are always up-to-date, the system is ready to install the latest versions of packages.

Key features:

- Modifies the `/etc/apt/apt.conf` file to enable periodic daily updates using the `ansible.builtin.lineinfile` module.

7) `setup_bash.yml`

Code 13 `setup_bash.yml` (Source: author)

```
roles > initial_setup > tasks > ! setup_bash.yml > {} 1
1  - name: Download bash setting user root
2    ansible.builtin.git:
3      repo: 'http://gitlab.kratto.cz/kratto/bash_settings.git'
4      dest: '/root/bash_settings'
5      single_branch: true
6      version: server
7      force: true
8
9  - name: Apply bash settings
10    ansible.builtin.command: '/root/bash_settings/move_settings.sh'
11
```

Applies custom bash settings for the root user by downloading a configuration repository and running a script to apply the settings.

Key features:

- Clones the bash settings repository using the `ansible.builtin.git` module and saves them to `/root/bash_settings` newly created directory.
- Executes a custom script (`move_settings.sh`) within the fetched directory to apply the settings.

8) secure_ssh.yml

Code 14 secure_ssh.yml (Source: author)

```
roles > initial_setup > tasks > ! secure_ssh.yml > ...
Ansible Tasks Schema - Ansible tasks file (ansible.json)

1  ---
2  - name: Allow root login only using key
3    ansible.builtin.lineinfile:
4      path: /etc/ssh/sshd_config
5      regexp: '^#PermitRootLogin '
6      line: PermitRootLogin prohibit-password
7      notify: Restart sshd service
8
9  - name: Allow login only using key
10   ansible.builtin.lineinfile:
11     path: /etc/ssh/sshd_config
12     regexp: '^#PubkeyAuthentication '
13     line: PubkeyAuthentication yes
14     notify: Restart sshd service
15
16  - name: Prohibit login only using password
17   ansible.builtin.lineinfile:
18     path: /etc/ssh/sshd_config
19     regexp: '^#PasswordAuthentication '
20     line: PasswordAuthentication no
21     notify: Restart sshd service
22
23  - name: Prohibit X forwarding
24   ansible.builtin.lineinfile:
25     path: /etc/ssh/sshd_config
26     regexp: '^#X11Forwarding '
27     line: X11Forwarding no
28     notify: Restart sshd service
29
30  - name: Do not print MOTD
31   ansible.builtin.lineinfile:
32     path: /etc/ssh/sshd_config
33     regexp: '^#PrintMotd '
34     line: PrintMotd no
35     notify: Restart sshd service
36
```

Applies security best practices to the SSH service.

Key features:

- Allows root login only with an SSH key.
- Prohibits password-based login.
- Disables X11 forwarding and suppresses the display of the warning banner.
- Notifies the Restart sshd service handler to apply changes.

9) add_root_ssh_keys.yml (legacy task)

The `add_root_ssh_keys.yml` task was initially part of the `initial_setup` role to ensure that all administrators from the department had access to new servers via their SSH keys. This task would add the required SSH keys to the root user's `~/.ssh/authorized_keys` file, granting administrative access to all authorized personnel.

However, this task is now largely redundant for virtual machines, as the SSH keys are directly stored in the VM template. For physical machines, where templates are not applicable, this task is still retained and can be used during deployments. Keeping the task file ensures that deployments remain consistent across environments that do not rely on VM templates.

Handlers

In Ansible, handlers are tasks that are only executed when explicitly notified by other tasks. They are typically used for actions that need to happen after a configuration change, such as restarting a service to apply new settings. It is a common convention in Ansible to place all handlers within a dedicated `handlers/` directory inside a role. This ensures better organization and reusability of the role components.

In `initial_setup` role we restart `ntp`, `rsyslog` and `sshd` services after reconfiguring them.

Code 15 Handlers in `initial_setup` role (Source: author)

```
roles > initial_setup > handlers > main.yml
1  - name: Restart ntp service
2    ansible.builtin.systemd:
3      state: restarted
4      name: systemd-timesyncd
5
6  - name: Restart rsyslog service
7    ansible.builtin.systemd:
8      state: restarted
9      name: rsyslog
10
11 - name: Restart sshd service
12   ansible.builtin.systemd:
13     state: restarted
14     name: sshd
15
```

ROLE: Java

This role installs a Java package, where a specific version can be set as a variable. Further, it configures the `JAVA_HOME` environment variable to ensure applications and tools can locate the Java runtime and development environment and in next step locks (holds) the installed version of the Java package. It is designed for systems where a specific version of

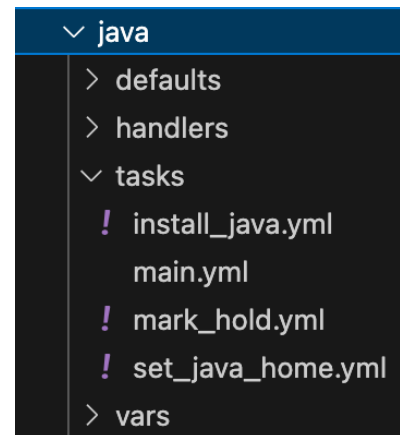
Java is required and might need to remain fixed to ensure compatibility with applications. If no specific version is defined, default-jdk is installed.

Code 16 Structure of java role
(Source: author)

Structure

The java role is located in the ansible/roles/java directory and consists of the following key components:

- Tasks: Three separate task files.
- Variables: The java_package variable can be defined to specify the Java package to install (for example openjdk-17-jdk). If this variable is not provided, the role defaults to installing default-jdk.



Code 17 Main.yml file in java role (Source: author)

```
roles > java > tasks > main.yml
1  ---
2  - name: Install Java package
3    ansible.builtin.import_tasks: install_java.yml
4
5  - name: Set JAVA_HOME environment variable and reload /etc/environment
6    ansible.builtin.include_tasks: set_java_home.yml
7
8  - name: Hold the "{{ java_package }}"
9    ansible.builtin.include_tasks: mark_hold.yml
10
```

Task files

1) install_java.yml

Code 18 install_java.yml (Source: author)

```
roles > java > tasks > ! install_java.yml > ...
Ansible Tasks Schema - Ansible tasks file (ansible.json)
1  ---
2  - name: Install Java package
3    ansible.builtin.apt:
4      name: "{{ java_package | default('default-jdk') }}"
5      state: present
6      update_cache: true
7
```

Installs the Java package specified by the java_package variable. If no specific package is defined, the role defaults to installing default-jdk.

Key features:

- Uses the `ansible.builtin.apt` module to ensure the package is present.
- Updates the package cache to ensure the latest version is installed.

2) `set_java_home.yml`

Code 19 `set_java_home.yml` (Source: author)

```
roles > java > tasks > ! set_java_home.yml > ...
Ansible Tasks Schema - Ansible tasks file (ansible.json)
1  ---
2  - name: Set JAVA_HOME environment variable
3    ansible.builtin.lineinfile:
4      path: /etc/environment
5      line: 'JAVA_HOME="/usr/lib/jvm/java-{{ java_package.split("-")[1] }}-openjdk-amd64"'
6      state: present
7      when: java_package != 'default-jdk'
8
9  - name: Reload environment variables
10    ansible.builtin.command:
11      . /etc/environment
12    changed_when: false
13    when: java_package != 'default-jdk'
14
```

Sets the `JAVA_HOME` environment variable and reloads the system environment to reflect the changes.

Key features:

- Adds the `JAVA_HOME` variable to `/etc/environment` using the `ansible.builtin.lineinfile` module.
- Dynamically generates the correct path based on the installed Java package.
- Reloads environment variables using the `ansible.builtin.command` module.
- Only executes when `java_package` is not set to `default-jdk`.

3) `mark_hold.yml`

Code 20 `mark_hold.yml` (Source: author)

```
roles > java > tasks > ! mark_hold.yml > ...
Ansible Tasks Schema - Ansible tasks file (ansible.json)
1  ---
2  - name: Hold the "{{ java_package }}"
3    ansible.builtin.dpkg_selections:
4      name: "{{ java_package }}"
5      selection: hold
6      when: java_package is defined
7
```

Holds the installed Java package version to prevent unintended updates.

Key features:

- Uses the `ansible.builtin.dpkg_selections` module to hold the Java package specified in the `java_package` variable.

ROLE: Tomcat

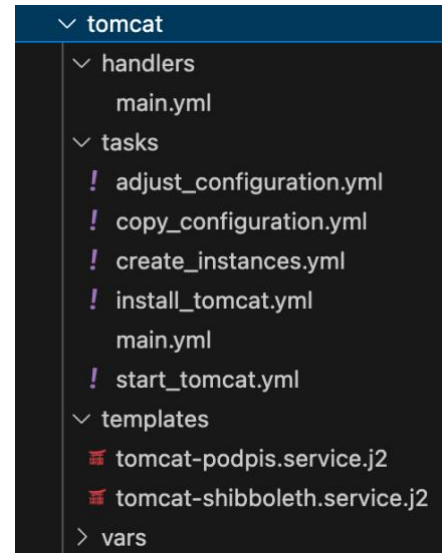
This role is used to install and configure Apache Tomcat, optionally enabling multiple instances. It provides flexibility in version selection, supports advanced configuration setups, and ensures that Tomcat is running as expected. The role is suitable for scenarios where specific Tomcat instances or customized configurations are required.

Structure

The `tomcat` role is located in the `ansible/roles/tomcat` directory and consists of the following key components:

- Tasks: Five separate task files.
- Handlers: Defined actions to restart services when required. Handlers in use are Restart tomcat and Restart systemd daemon.
- Variables: Used for modularity and adaptability of the role. Variables in the `tomcat` role are `tomcat_version`, `tomcat_instances`, `tomcat_shibboleth_source`, `tomcat_podpis_source`.
 - `Tomcat_instances` must have these parameters defined: `name`, `group`, `instance`, `template`.
- Templates: Jinja2-based files used to dynamically generate configuration files or scripts during role execution. They are commonly used in roles to create systemd service files, application configuration files, or other scripts tailored to the specific requirements of the deployment environment. In `tomcat` role we can find two templates for the instance's systemd service configurations.

Code 21 Structure of tomcat role
(Source: author)



Code 22 Template in tomcat role (Source: author)

```
roles > tomcat > templates > tomcat-shibboleth.service.j2
1  [Unit]
2  Description=Tomcat {{ tomcat_version | default('10') }} servlet container for {{ item.name }}
3  After=network.target
4
5  [Service]
6  Type=forking
7
8  Environment="JAVA_HOME=/usr/lib/jvm/{{ java_home | default('default-java') }}"
9  Environment="CATALINA_PID=/opt/tomcat-{{ item.instance }}/temp/tomcat.pid"
10 Environment="CATALINA_HOME=/usr/share/tomcat{{ tomcat_version | default('10') }}"
11 Environment="CATALINA_BASE=/opt/tomcat-{{ item.instance }}"
12 Environment="CATALINA_OPTS=-Xms512M -Xmx1556M -server -XX:+UseParallelGC"
13 Environment="JAVA_OPTS=-Djava.awt.headless=true"
14
15 ExecStart=/opt/tomcat-{{ item.instance }}/bin/startup.sh
16 ExecStop=/opt/tomcat-{{ item.instance }}/bin/shutdown.sh
17
18 User={{ item.name }}
19 Group={{ item.group }}
20 UMask=0007
21 RestartSec=10
22 Restart=always
23
24 [Install]
25 WantedBy=multi-user.target
```

Templates allow the inclusion of variables, loops, and conditionals to customize the generated files based on the target system's parameters or role variables.

Code 23 Main.yml file in tomcat role (Source: author)

```
roles > tomcat > tasks > main.yml
1  ---
2  - name: Install Tomcat
3    ansible.builtin.include_tasks: install_tomcat.yml
4
5  - name: Create separate tomcat instances
6    ansible.builtin.include_tasks: create_instances.yml
7    when: tomcat_instances is defined
8
9  - name: Copy existing Tomcat configuration
10   ansible.builtin.include_tasks: copy_configuration.yml
11   tags:
12     - never
13     - id
14
15  - name: Adjust versioning in configuration
16   ansible.builtin.include_tasks: adjust_configuration.yml
17   tags:
18     - never
19     - id
20
21  - name: Start Tomcat service
22   ansible.builtin.include_tasks: start_tomcat.yml
23
```

Task files

1) install_tomcat.yml

Code 24 install_tomcat.yml (Source: author)

```
roles > tomcat > tasks > ! install_tomcat.yml > ...
Ansible Tasks Schema - Ansible tasks file (ansible.json)
1  ---
2  - name: Install Tomcat
3    ansible.builtin.apt:
4      name: "{{ item }}"
5      state: present
6    with_item: "tomcat{{ tomcat_version | default('10') }}"
7    when: tomcat_instances is not defined
8
9  - name: Install Tomcat-user
10    ansible.builtin.apt:
11      name: "{{ item }}"
12      state: present
13    with_item: "tomcat{{ tomcat_version | default('10') }}-user"
14    when: tomcat_instances is defined
15
```

This task installs Apache Tomcat along with the tomcat-user package if multiple instances are required and defined.

Key features:

- Installs the default Tomcat version (tomcat10) or a specified version using the tomcat_version variable.
- Supports installing the tomcat-user package for creating multiple instances.
- Handles conditional installation based on whether variable tomcat_instances is defined.

2) create_instances.yml

Code 25 create_instances.yml, first part (Source: author)

```
roles > tomcat > tasks > ! create_instances.yml > ...
Ansible Tasks Schema - Ansible tasks file (ansible.json)

1  ---
2  - name: Gather the package facts
3    ansible.builtin.package_facts:
4      manager: auto
5
6  # pexpect needed for ansible.builtin.expect module
7  - name: Install pexpect
8    ansible.builtin.apt:
9      update_cache: true
10     name:
11       - python3-pexpect
12     when: " 'python3-pexpect' not in ansible_facts.packages"
13
14  - name: Create users for tomcat instances
15    ansible.builtin.user:
16      name: "{{ item.name }}"
17      shell: /bin/bash
18      home: /home/{{ item.name }}
19      with_items: "{{ tomcat_instances }}"
20
21  - name: Create groups for tomcat instances
22    ansible.builtin.group:
23      name: "{{ item.group }}"
24      with_items: "{{ tomcat_instances }}"
25
26  - name: Remove /opt/tomcat-shibboleth if it exists
27    ansible.builtin.file:
28      path: /opt/tomcat-shibboleth
29      state: absent
30
31  - name: Remove /opt/tomcat-podpis if it exists
32    ansible.builtin.file:
33      path: /opt/tomcat-podpis
34      state: absent
35
```

Code 26 create_instances.yml, second part (Source: author)

```
35
36  - name: Create tomcat instances
37    ansible.builtin.expect:
38      command: tomcat{{ tomcat_version | default('10') }}-instance-create /opt/tomcat-{{ item.instance }}
39      responses:
40        "Type <ENTER> to continue, <CTRL-C> to abort.": ''
41        timeout: 300
42      become: true
43      become_user: "{{ item.name }}"
44      with_items: "{{ tomcat_instances }}"
45
46  - name: Create tomcat systemd service file for instances
47    ansible.builtin.template:
48      src: "{{ item.template }}"
49      dest: /etc/systemd/system/tomcat-{{ item.instance }}.service
50      mode: '0644'
51      with_items: "{{ tomcat_instances }}"
52      notify: Reload systemd daemon
53
```

This task creates custom Tomcat instances with unique users, groups, and directories, and sets up systemd service files.

Key features:

- Gathers package facts and ensures pexpect is installed as it is needed for ansible.builtin.expect module.
- Ansible.builtin.expect used for installation, where user is asked questions.
- Runs the tomcat-instance-create command for creating Tomcat instances only if the variable is defined.
- Creates users and groups for each instance defined in tomcat_instances.
- Configures and applies systemd service files for each instance via templates.

3) copy_configuration.yml

This task handles copying and applying custom configurations to Tomcat instances for our specific need on id servers. For this reason we have to apply `--tags id` to import this task file.

The custom configurations for applying Tomcat instances in our specific environment are tailored to the needs of our authentication servers. These configurations will be stored in our internal GitLab repository, where they can be accessed and updated by authorized team members. Since each instance might require unique configurations, there is no need to share the task file externally.

Key features:

- Synchronizes tomcat configurations from defined sources to the target directories using variables to select source directories.
- Configures ownership, permissions, and proper directory structure for Tomcat instances.
- Provides flexibility for users to apply their own custom configurations.

4) adjust_configuration.yml

Code 27 adjust_configuration.yml (Source: author)

```
roles > tomcat > tasks > ! adjust_configuration.yml > ...
1  ---
2  - name: Replace tomcat9 with tomcat{{ tomcat_version | default('10') }} in startup script
3    ansible.builtin.replace:
4      path: "/opt/tomcat-{{ item.instance }}/bin/startup.sh"
5      regexp: 'tomcat9'
6      replace: "tomcat{{ tomcat_version | default('10') }}"
7      with_items: "{{ tomcat_instances }}"
8
9  - name: Replace tomcat9 with tomcat{{ tomcat_version | default('10') }} in shutdown script
10   ansible.builtin.replace:
11     path: "/opt/tomcat-{{ item.instance }}/bin/shutdown.sh"
12     regexp: 'tomcat9'
13     replace: "tomcat{{ tomcat_version | default('10') }}"
14     with_items: "{{ tomcat_instances }}"
15
16  - name: Replace tomcat9 with tomcat{{ tomcat_version | default('10') }} in setenv script
17   ansible.builtin.replace:
18     path: /opt/tomcat-podpis/bin/setenv.sh
19     regexp: 'tomcat9'
20     replace: "tomcat{{ tomcat_version | default('10') }}"
21
```

This task ensures compatibility by updating our old scripts to use the new specified Tomcat version. Specific for id servers, for this reason we have to apply `-tags id` to import this task file.

5) start_tomcat.yml

Code 28 start_tomcat.yml (Source: author)

```
roles > tomcat > tasks > ! start_tomcat.yml > ...
Ansible Tasks Schema - Ansible tasks file (ansible.json)
1  ---
2  # default for tomcat10 only
3  - name: Start Tomcat service
4    ansible.builtin.service:
5      name: "tomcat{{ tomcat_version | default('10') }}"
6      state: started
7      enabled: true
8      when: tomcat_instances is not defined
9
10 - name: Enable tomcat-user services
11   ansible.builtin.systemd:
12     name: "tomcat-{{ item.instance }}"
13     enabled: true
14     state: started
15   with_items: "{{ tomcat_instances }}"
16   when: tomcat_instances is defined
17
```

This task manages the starting and enabling of Tomcat services, including multiple instances.

Key features:

- Starts and enables the default tomcat10 service or version of tomcat defined in tomcat_version variable when no custom instances are defined.
- Enables and starts custom systemd services for each instance defined in tomcat_instances.

Code 29 Handlers in tomcat role (Source: author)

```
roles > tomcat > handlers >    main.yml
1  - name: Restart tomcat
2    ansible.builtin.systemd:
3      name: tomcat
4      state: restarted
5
6  - name: Reload systemd daemon
7    ansible.builtin.systemd:
8      name: daemon
9      state: reloaded
10
```

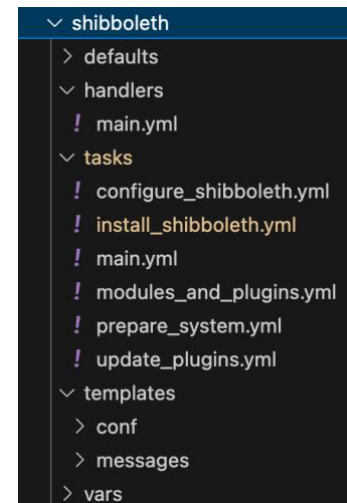
ROLE: Shibboleth

This role automates the deployment and configuration of a Shibboleth Identity Provider (IdP). This includes setting up the required directories, downloading and installing the Shibboleth package, applying custom configurations, and enabling necessary modules and plugins. The role ensures that the IdP instance is configured securely and consistently, adhering to best practices for access control, attribute release policies, and module management.

Structure

The shibboleth role is located in the `ansible/roles/shibboleth` directory and consists of the following key components:

- **Tasks:** Four separate task files.
- **Variables:** The Shibboleth role uses a set of variables to ensure modularity and adaptability. These variables allow customization of key parameters such as installation paths, user configurations, plugin lists, LDAP settings, and credentials.
- **Templates:** Jinja2 templates that are used to generate the configuration files for the Shibboleth IdP deployment. These templates provide a modular and dynamic way to manage Shibboleth configurations, allowing customization and flexibility by integrating variable values defined in the Ansible role.



Code 31 Main.yml file in shibboleth role (Source: author)

```
roles > shibboleth > tasks > ! main.yml > ...
Ansible Tasks Schema - Ansible tasks file (ansible.json)
1  ---
2  - name: Prepare directory and download & untar Shibboleth IdP package
3    ansible.builtin.include_tasks: prepare_system.yml
4
5  - name: Install Shibboleth IdP
6    ansible.builtin.include_tasks: install_shibboleth.yml
7
8  - name: Configure Shibboleth IdP config files
9    ansible.builtin.include_tasks: configure_shibboleth.yml
10
11 - name: Enable modules and plugins
12   ansible.builtin.include_tasks: modules_and_plugins.yml
13
```

Task files

1) prepare_system.yml

Code 32 prepare_system.yml (Source: author)

```
roles > shibboleth > tasks > ! prepare_system.yml > {} 1 > {} unarchive
1  ---
2  - name: Create /opt/shibboleth directory
3    ansible.builtin.file:
4      path: "{{ shibboleth_idp_home }}"
5      state: directory
6
7  - name: Download and untar Shibboleth IdP package
8    unarchive:
9      src: "https://shibboleth.net/downloads/identity-provider/
10         {{ shibboleth_idp_version | default('latest5') }}/shibboleth-identity-provider-{{ shibboleth_idp_version | default('5.1.3') }}.tar.gz"
11      dest: "{{ shibboleth_idp_home }}"
12      remote_src: yes
13
```

This task prepares the system for the deployment of Shibboleth IdP by creating necessary directories and downloading the required Shibboleth IdP package.

Key features:

- Creates the required directory /opt/shibboleth, which is used for storing Shibboleth related files, if it does not already exist. The directory path is configurable using the shibboleth_idp_home variable.
- Downloads and untars the Shibboleth IdP package from the official Shibboleth repository. The version of the package is defined by the shibboleth_idp_version variable, which defaults to version 5.1.3 if not specified.
- Supports remote file extraction, ensuring the Shibboleth IdP package is retrieved and extracted directly on the target server.

2) Install_shibboleth.yml

Code 33 install_shibboleth.yml (Source: author)

```
roles > shibboleth > tasks > ! install_shibboleth.yml > {} 1 > {} ansible.builtin.file
1  ---
2  - name: Run installation script
3    ansible.builtin.expect:
4      command: "sh {{ shibboleth_idp_home }}/shibboleth-identity-provider-{{ shibboleth_idp_version | default('5.1.3') }}/bin/install.sh"
5      responses:
6        Installation Directory: "{{ shibboleth_idp_home }}"
7        Host Name: "{{ hostname }}"
8        SAML EntityID: "{{ entityID }}"
9        Attribute Scope: "{{ idp_scope }}"
10
11 - name: Recursively remove archive # only needed for installation
12   ansible.builtin.file:
13     path: "{{ shibboleth_idp_home }}/shibboleth-identity-provider-{{ shibboleth_idp_version | default('5.1.3') }}"
14     state: absent
15
```

This task handles the installation of Shibboleth Identity Provider (IdP) by running the official installation script and cleaning up unnecessary files after the installation process.

Key features:

- Runs the installation script (install.sh), which is provided by the Shibboleth IdP package. It automates the setup of the Shibboleth IdP, with responses provided for configuration prompts:
 - Installation Directory: The directory where Shibboleth IdP will be installed (variable shibboleth_idp_home).
 - Host Name: The server's hostname (variable hostname).
 - SAML EntityID: The entity ID used for SAML assertions (variable entityID).
 - Attribute Scope: The scope for identity attributes (variable idp_scope).
- Cleans up after installation by recursively removing the installation archive and related files, which are no longer necessary once the installation is complete. The target directory is removed to prevent leftover files from consuming disk space.

3) configure_shibboleth.yml

This task file is used to configure Shibboleth IdP with specific settings and custom templates tailored to the needs of the university's infrastructure.

Key considerations:

- The task file contains highly specific configurations that are tied to the university's unique requirements, making it less applicable for general use.
- Most configurations in this task file adjust Shibboleth templates found in a clean installation. For general use, these files can be manually adjusted or replaced with custom templates based on the default ones provided during the installation process.
- To ensure security and maintain internal standards, these configurations are saved privately in the university's GitLab repository as documentation as code. Sharing this role in a public Git repository requires prior evaluation to address potential security risks, which is something I plan to achieve in the near future.

Key features:

- Ensures that all Shibboleth related files have appropriate ownership, group settings, and permissions to maintain security and access control.
- Uses a variety of custom dynamic templates to configure Shibboleth, allowing flexibility.

Rationale for internal use only:

- This task file reflects sensitive and specific configurations for the university's infrastructure.
- Some templates are based on internal standards and policies, which could reveal operational details if shared prematurely.

- The role is designed for generalization, but adjustments are necessary for other environments to account for differing infrastructure and security needs.

Future plans for public sharing:

- I intend to share the created roles in a public Git repository once a thorough evaluation of security risks is completed.
- Before sharing, sensitive configurations will be stripped, and documentation will provide guidance on creating custom templates for general use.

4) modules_and_plugins.yml

Code 34 modules_and_plugins.yml, first part (Source: author)

```
roles > shibboleth > tasks > ! modules_and_plugins.yml > ...
Ansible Tasks Schema - Ansible tasks file (ansible.json)

1  ---
2  - name: Remove commons-logging-api-1.1.jar
3    ansible.builtin.file:
4      path: /opt/shibboleth-idp/edit-webapp/WEB-INF/lib/commons-logging-api-1.1.jar
5      state: absent
6
7  - name: Download PostgreSQL JDBC driver
8    ansible.builtin.get_url:
9      url: https://jdbc.postgresql.org/download/postgresql-42.7.3.jar
10     dest: /opt/shibboleth-idp/edit-webapp/WEB-INF/lib/postgresql-42.7.3.jar
11     owner: shibboleth-idp
12     group: shibboleth-idp
13
14  - name: Download MySQL JDBC driver
15    ansible.builtin.get_url:
16      url: https://cdn.mysql.com/Downloads/Connector-J/mysql-connector-j_9.0.0-1debian12_all.deb
17      dest: /opt/shibboleth-idp/edit-webapp/WEB-INF/lib/mysql-connector-j_9.0.0-1debian12_all.deb
18      owner: shibboleth-idp
19      group: shibboleth-idp
20
21  - name: Install MySQL JDBC driver package
22    ansible.builtin.command: dpkg -i /opt/shibboleth-idp/edit-webapp/WEB-INF/lib/mysql-connector-j_9.0.0-1debian12_all.deb
23    ignore_errors: true
24
25  - name: Copy MySQL JDBC driver file to /opt/shibboleth-idp
26    ansible.builtin.copy:
27      src: /usr/share/java/mysql-connector-j-9.0.0.jar
28      dest: /opt/shibboleth-idp/edit-webapp/WEB-INF/lib/mysql-connector-j-9.0.0.jar
29      owner: shibboleth-idp
30      group: shibboleth-idp
31      mode: '0644'
32      remote_src: yes
33
34  - name: Download Jakarta JSTL api
35    ansible.builtin.get_url:
36      url: https://repo1.maven.org/maven2/jakarta/servlet/jsp/jstl-api/3.0.0/jakarta.servlet.jsp.jstl-api-3.0.0.jar
37      dest: /opt/shibboleth-idp/edit-webapp/WEB-INF/lib/jakarta.servlet.jsp.jstl-api-3.0.0.jar
38      owner: shibboleth-idp
39      group: shibboleth-idp
40
41  - name: Install '{{item.name}}' plugin
42    ansible.builtin.expect:
43      command: ./plugin.sh -I '{{item.idp_name}}'
44      responses:
45        "Accept this key:": 'y'
46      args:
47        chdir: /opt/shibboleth-idp/bin
48      ignore_errors: true
49      with_items: "{{ plugins }}"
50
```

Code 35 modules_and_plugins.yml, second part (Source: author)

```
50
51 - name: Test if idp.admin.Hello module is enabled
52   ansible.builtin.command: bin/module.sh -t idp.admin.Hello
53   args:
54     chdir: /opt/shibboleth-idp/
55     register: module_test_result
56     ignore_errors: true
57
58 - name: Enable idp.admin.Hello module if not already enabled
59   ansible.builtin.command: bin/module.sh -e idp.admin.Hello
60   args:
61     chdir: /opt/shibboleth-idp/
62     when: module_test_result.rc != 0
63
```

This task file handles the configuration of modules and plugins required for the Shibboleth IdP and manages necessary dependencies like JDBC drivers.

Key features:

- Dependency management:
 - Removes outdated or unnecessary libraries
 - Downloads and installs the required PostgreSQL and MySQL JDBC drivers to ensure proper database connectivity.
 - Installs the Jakarta JSTL API for advanced servlet and JSP support.
- Module and plugin setup:
 - Configures Shibboleth plugins dynamically using the plugin.sh script.
 - Tests and enables the idp.admin.Hello module.

Code 36 Templates in shibboleth role (Source: author)

